

SLAPP: Poisoning Prevention in Federated Learning and Differential Privacy via Stateful Proofs of Execution

Norrathep Rattanavipanon* and Ivan De Oliveira Nunes†

*College of Computing, Prince of Songkla University

†SPINS Group, Department of Informatics, University of Zurich

Abstract—The rise of IoT-driven distributed data analytics, coupled with increasing privacy concerns, has led to a demand for effective privacy-preserving and federated data collection/model training mechanisms. In response, approaches such as Federated Learning (FL) and Local Differential Privacy (LDP) have been proposed and attracted much attention over the past few years. However, they still share the common limitation of being vulnerable to poisoning attacks wherein adversaries compromising edge devices feed forged (a.k.a. “poisoned”) data to aggregation back-ends, undermining the integrity of FL/LDP results.

In this work, we propose a system-level approach to remedy this issue based on a novel security notion of Proofs of Stateful Execution (PoSX) for IoT/embedded devices’ software. To realize the PoSX concept, we design SLAPP: a System-Level Approach for Poisoning Prevention. SLAPP leverages commodity security features of embedded devices – in particular ARM TrustZone-M security extensions – to verifiably bind raw sensed data to their correct usage as part of FL/LDP edge device routines. As a consequence, it offers robust security guarantees against poisoning. Our evaluation, based on real-world prototypes featuring multiple cryptographic primitives and data collection schemes, showcases SLAPP’s security and low overhead.

I. INTRODUCTION

With the rise of IoT and distributed big data analytics, data produced by edge devices have become increasingly important to understand users’ behaviors, enhance the user experience, and improve the quality of service. At the same time, privacy concerns have scaled significantly fueled by the collection (or leakage) of sensitive user data [28], [35], [73], [29], [32]. To reconcile privacy and utility, several mechanisms have been proposed to enable efficient and privacy-preserving collection of data produced by (typically resource-constrained) edge IoT devices. For example, Google has proposed Federated Learning (FL) aiming to collect and train models based on user data in a distributed, lightweight, and privacy-preserving fashion [34]; Microsoft employs a mechanism based on Local Differential Privacy (LDP) to collect statistics of sensitive telemetry data across millions of devices [19].

As illustrated in Fig. 1, a crucial security obstacle in the adoption of these schemes is an adversary ($\mathcal{Adv}$) that feeds back-end aggregators with forged data, sabotaging the entire collection process even when the other participants (other edge devices and the back-end) are honest. These attacks, known as *poisoning attacks*, have been widely recognized and are currently considered a major problem for both FL [66] and LDP-based data collection mechanisms [7]. In particular, embedded/IoT devices are highly susceptible to software exploits that potentially lead to these attacks due to their inherent lack of security mechanisms [2], [69]. For instance, $\mathcal{Adv}$ can exploit memory-safety vulnerabilities such as buffer

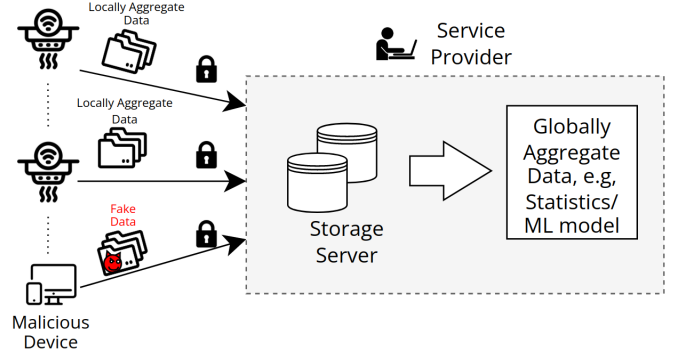


Fig. 1: Poisoning in FL/LDP-based systems

overflows (e.g., CVE-2020-10023 [47]) or heap-based exploits (e.g., CVE-2017-14201 [46]) to gain remote code execution power; this in turn enables poisoning attacks directly on these devices. Moreover, IoT ecosystems are often deployed as *monocultures* [69], where the same types of devices (possibly containing the same exploitable vulnerability) are deployed by service providers. This intensifies the risk of poisoning attacks as $\mathcal{Adv}$ can compromise multiple IoT devices simultaneously (via the same exploit), facilitating poisoning on a large scale.

Existing mitigation techniques are either data-driven (e.g., in the case of data poisoning detection [7], [6], [37], [23]) or algorithmic, e.g., by making FL and LDP mechanisms more resilient against these attacks [4], [26], [24], [31], [62]. In both cases, security is by design best-effort, since these approaches cannot detect/prevent data poisoning at its source, i.e., at the edge devices themselves (typically, resource-constrained IoT devices).

In this work, we propose a systematic and practical treatment of data poisoning at its source by leveraging architectural security features of contemporary embedded systems. This ensures that raw sensed data is correctly linked with its respective local processing, ultimately generating local aggregated data whose integrity and authenticity can be verified by back-ends in FL/LDP-based applications.

Specifically, we build upon the recently introduced concept of Proofs of Execution (PoX) [13] for simple embedded systems. PoX allows a low-end embedded device – Prover ($\mathcal{Prv}$) – to convince a remote Verifier ($\mathcal{Vrf}$) that a specific function $\mathcal{F}$ has been executed successfully (i.e., from its first to its last instruction) on $\mathcal{Prv}$. Furthermore, PoX binds obtained results (or outputs) to a timely instance of this execution. A similar notion [39] has also been explored in high-end systems (e.g., general-purpose computers and servers, as opposed to embedded devices) based on trusted platform modules

(TPMs) [68].

Intuitively, PoX can convey if the result received by an FL/LDP back-end (local aggregate data) truly originates from an edge device that has obtained this data through the correct execution of the expected software, thus thwarting poisoning attacks. On the other hand, as detailed next, the adoption of PoX in FL/LDP-based mechanisms introduces unique non-trivial challenges that require re-thinking and re-designing existing PoX methods for this purpose.

A. On the Insufficiency of Classic PoX to Avert Poisoning

PoX is a challenge-response protocol composed of the following steps:

- 1) $\mathcal{Vrf}$ sends an authenticated request containing a cryptographic challenge ($\mathcal{Chal}$) and asking $\mathcal{Prv}$ to execute $\mathcal{F}$.
- 2) $\mathcal{Prv}$ authenticates the request, executes $\mathcal{F}$ obtaining output $\mathcal{O}$, and generates a cryptographic proof σ of this execution by measuring (signing or a MAC-ing) $\mathcal{F}$'s implementation in program memory along with received $\mathcal{Chal}$, produced $\mathcal{O}$, and execution metadata that conveys to $\mathcal{Vrf}$ if $\mathcal{F}$ execution was performed correctly.
- 3) $\mathcal{Prv}$ returns the output and proof (σ , $\mathcal{O}$) to $\mathcal{Vrf}$.
- 4) $\mathcal{Vrf}$ verifies whether σ corresponds to the expected $\mathcal{F}$ code, received output $\mathcal{O}$, and expected execution metadata. If so, it concludes that $\mathcal{F}$ has executed successfully on $\mathcal{Prv}$ with result $\mathcal{O}$.

Step 2 above must be securely implemented by a root of trust (RoT) within $\mathcal{Prv}$ to ensure (1) temporal consistency between $\mathcal{F}$'s measurement and its execution, (2) correctness of $\mathcal{F}$ execution and generated $\mathcal{O}$ at run-time, and (3) confidentiality of the cryptographic secret used to compute σ . This RoT implementation must be unmodifiable, even when $\mathcal{Prv}$'s application software is fully compromised. The latter is typically obtained through hardware support, e.g., from ARM TrustZone [55] (see Section II-B) or similar mechanisms.

We observe that the aforementioned PoX notion has important practical limitations. It assumes $\mathcal{F}$ to be: (1) inputless, i.e., $\mathcal{F}$ cannot depend on inputs external to $\mathcal{Prv}$, and (2) stateless, i.e., $\mathcal{F}$ must not depend on states produced by prior PoX instances in $\mathcal{Prv}$. As a result, PoX is only suitable for simple self-contained programs that may process locally collected data (via $\mathcal{Prv}$ local I/O interfaces) but do not depend on external inputs or prior execution states. This impedes the application of PoX to FL/LDP-based mechanisms.

For FL integrity, $\mathcal{Prv}$ should prove that a training function $\mathcal{F}$ was executed on local training dataset D using $\mathcal{Vrf}$ -supplied global weights W and training parameters ($\mathcal{Vrf}$ in this setting is the service provider). Moreover, no portion of D should be revealed to $\mathcal{Vrf}$, requiring multiple PoX instances (e.g., multiple sensing routines executed over time) to correctly produce all data points in D . As detailed in Section II-A, similar requirements exist in LDP algorithms. Therefore, standard PoX can not be applied in these settings.

B. Our Contributions

To address the aforementioned limitations, this work introduces the security notion of Proof of Stateful Execution

(PoSX) to enable **input validation** and **state preservation**, in addition to classic PoX guarantees. The former relaxes the constraint of inputless functions in traditional PoX, while the latter ensures that PoSX can use $\mathcal{Prv}$ pre-existing states as long as they originate from a prior authentic PoSX execution. In essence, PoSX offers assurance that execution of $\mathcal{F}$, computed with authentic input $\mathcal{I}$ and state $\mathcal{S}$, denoted $\mathcal{F}_{\mathcal{S}}(\mathcal{I})$, occurred faithfully, without disclosing $\mathcal{S}$ to $\mathcal{Vrf}$.

To realize PoSX on real-world IoT settings, we design and implement SLAPP: a System-Level Approach for Poisoning Prevention. SLAPP's design and security rely on the commercially available TrustZone-M Security Extension, widely present even in low-end embedded devices: those based on ARM Cortex-M Micro-Controller Units (MCUs). This facilitates the immediate real-world implementation of SLAPP onto current IoT devices.

We show that SLAPP can support a wide range of data collection schemes, including FL and LDP, all with poisoning-free guarantees. Compared to prior data-centric mitigations in FL [62], [6], [38] or LDP [7], SLAPP offers two key benefits: as a system-level approach, it is agnostic to the underlying collection scheme (and implementation thereof), thereby capable of supporting both FL and LDP without changes in its trusted computing base (TCB), i.e., its TCB remains the same for any function $\mathcal{F}$. Secondly, it primarily operates on the $\mathcal{Prv}$ -side while requiring one additional verification operation on $\mathcal{Vrf}$. This makes SLAPP complementary to many server-side techniques, allowing seamless integration and the ability to further benefit from these techniques. We elaborate on these points in Section X. In summary, our contributions are:

- 1) **New PoSX Security Notion:** we define a new security primitive, called Proof of Stateful Execution (PoSX). PoSX retains the same guarantees as classic PoX while addressing its limitations of inputless and stateless execution and maintaining privacy of underlying execution states *vis-a-vis* $\mathcal{Vrf}$.
- 2) **Practical Poisoning Prevention:** We develop SLAPP: a design to realize PoSX that is applicable to resource-constrained embedded devices. We integrate SLAPP with FL and LDP implementations to support poison-free instantiations of these algorithms without loss of privacy.
- 3) **Real-World Prototypes:** To validate SLAPP and foster reproducibility, we provide three implementation variants, each utilizing distinct cryptographic schemes: symmetric, traditional asymmetric, and quantum-resistant primitives. These implementations are prototyped and open-sourced [57] on a real-world IoT development board: NUCLEO-L552ZE-Q [61].
- 4) **Evaluation:** We conduct various experiments to assess SLAPP's efficiency. Our results demonstrate small run-time and memory overhead atop the baseline, in exchange for increased security and flexibility. Finally, we provide detailed case studies highlighting SLAPP's efficiency and efficacy in thwarting poisoning attacks within FL/LDP-based mechanisms.

II. BACKGROUND

A. Privacy-Preserving Data Collection Schemes

In this section, we overview FL and LDP. We use the notation ϕ to indicate an empty/null variable.

Local Differential Privacy-based Data Collection (LDP-DC). The goal of this scheme is for an IoT sensor, $\mathcal{P}rv$, to output a noisy sensor value $\mathcal{O}$ such that it preserves ϵ -LDP; informally speaking, ϵ -LDP guarantees that $\mathcal{O}$ leaks no information about its original value $\mathcal{O}'$ except with a small probability constrained by ϵ . At the same time, collecting $\mathcal{O}$ from a number of $\mathcal{P}rv$ -s allows $\mathcal{V}rf$ to obtain certain statistics of the collected sensor values with high confidence. Various LDP mechanisms have been proposed to achieve ϵ -LDP in different settings. As an example case, we focus on the LDP mechanism called Basic RAPPOR [22] noting that the concepts introduced in this work can be generalized to other LDP-based mechanisms.

LDP-DC, as detailed in Algorithm 1, describes how $\mathcal{P}rv$ performs data collection using Basic RAPPOR. First, $\mathcal{P}rv$ collects a sensor reading $\mathcal{O}'$ by calling the sensor function $\mathcal{F}$ on a given input $\mathcal{I}$, i.e., $\mathcal{F}(\mathcal{I})$. Assuming $\mathcal{O}'$ can be represented using k -bit unsigned integer, $\mathcal{P}rv$ next performs a unary encoding (UE) that transforms $\mathcal{O}'$ into a 2^k -bit vector b in which only the $\mathcal{O}'^{th}$ bit is set to 1 and other bits are 0.

Algorithm 1: Implementation of LDP-DC on $\mathcal{P}rv$

Input : Sensor function $\mathcal{F}$, Input to sensor function $\mathcal{I}$, Basic RAPPOR parameters f, p and q
Output: Noisy sensor output $\mathcal{O}$
State : PRR mapping B

```

1 Func LDP-DC( $\mathcal{F}, \mathcal{I}, f, p, q$ ):
2    $\mathcal{O}' \leftarrow \mathcal{F}(\mathcal{I})$ 
3    $b \leftarrow UE(\mathcal{O}')$ 
4    $b' \leftarrow PRR(b, f, B)$ 
5    $\mathcal{O} \leftarrow IRR(b', p, q)$ 
6   return  $\mathcal{O}$ 

```

Next, on input b and parameter f , $\mathcal{P}rv$ invokes Permanent Randomized Response (PRR) function and produces a 2^k -bit noisy vector b' as output, where b'_i – the i^{th} bit of b' – is computed as:

$$b'_i = \begin{cases} 1, & \text{with probability } \frac{f}{2} \\ 0, & \text{with probability } \frac{1-f}{2} \end{cases} \quad (1)$$

Once b' is generated, $\mathcal{P}rv$ caches an input-output mapping (b, b') to a local state variable B (i.e., $B[b] = b'$) and, for all future encounters of the same input b , returns $B[b]$ without recomputing the entire PRR function.

Finally, on input b' and parameters p and q , $\mathcal{P}rv$ executes Instantaneous Randomized Response (IRR) function that returns a 2^k -bit binary vector $\mathcal{O}$ to $\mathcal{V}rf$ such that:

$$\mathbb{P}(\mathcal{O}_i = 1) = \begin{cases} p, & \text{if } b'_i = 1 \\ q, & \text{otherwise} \end{cases} \quad (2)$$

Using LDP-DC, $\mathcal{V}rf$ can estimate $\tilde{f}_x$, the frequency of sensor value x by:

$$\tilde{f}_x = \frac{c_x - (q + \frac{1}{2}fp - \frac{1}{2}fq)n}{(1-f)(p-q)n} \quad (3)$$

where c_x represents the number of reports that have x^{th} bit set and n is the number of reports received by $\mathcal{V}rf$. We refer the interested reader to the Basic RAPPOR paper [22] for details on how to select f, p, q to satisfy ϵ -LDP.

We emphasize that the state variable B serves as a critical component to the privacy of this scheme. To ensure ϵ -LDP, B must be accurately updated in the current Basic RAPPOR session and its updated value must be carried forward to the subsequent session. Moreover, B must be oblivious to $\mathcal{V}rf$; otherwise, $\mathcal{V}rf$ can reverse the PRR operation to recover the original sensor reading $\mathcal{O}'$. Also, this scheme requires input arguments f, p , and q as part of its execution. As noted earlier, however, standard PoX does not support execution using external input arguments or $\mathcal{P}rv$ local states that must be oblivious to $\mathcal{V}rf$.

Algorithm 2: Implementation of FL-DC on $\mathcal{P}rv$

Input : Sensor function $\mathcal{F}$, Input to sensor function $\mathcal{I}$
Output: ϕ
State : Local dataset D

```

1 Func Sense-Store( $\mathcal{F}, \mathcal{I}$ ):
2    $\mathcal{O} \leftarrow \mathcal{F}(\mathcal{I})$ 
3    $D.append(\mathcal{O})$ 

Input : Globally trained weights  $W$ , number of epochs  $t$ , learning rate  $\alpha$   

Output: Locally trained weights  $\mathcal{O}$   

State :  $D$ 
```

```

4 Func Train( $W, t, \alpha$ ):
5   for  $k \leftarrow 1$  to  $t$  do
6     //  $\nabla$  is a gradient function
7      $W \leftarrow W - \alpha \cdot \nabla(W; D)$ 
8   end
9    $\mathcal{O} \leftarrow W$ 
10  return  $\mathcal{O}$ 

```

Federated Learning-based data collection (FL-DC). Contrary to LDP-DC, FL-DC [34] requires $\mathcal{P}rv$ to send a locally trained machine learning (ML) partial model to $\mathcal{V}rf$ instead of sensor readings. This keeps raw sensor data local to $\mathcal{P}rv$ and not directly accessible by $\mathcal{V}rf$.

A typical FL-DC consists of two phases depicted in Algorithm 2. In the first phase, $\mathcal{P}rv$ invokes the Sense-Store function to collect a raw sensor reading and store it in a local list D . This phase can be repeated multiple times over a certain period to collect more training data on $\mathcal{P}rv$. Once sufficient training data is gathered, $\mathcal{P}rv$ receives globally trained weights W from $\mathcal{V}rf$ along with the training parameters (t and α). It then triggers the Train function that utilizes W as a base model to train the data in D . This function outputs the locally trained weights $\mathcal{O}$ without disclosing D to $\mathcal{V}rf$.

After collecting weights $\mathcal{O}$ -s from multiple devices, $\mathcal{V}rf$ can aggregate them using several methods, e.g., FedAvg [34] averages the received weights and sets the result as the new global weights: $W \leftarrow W + \eta \cdot \sum_{i=1}^m (\mathcal{O}_i - W)/m$ for some global learning rate η .

Similar to LDP-DC, executing FL-DC functions relies on both input arguments W, t and α as well as $\mathcal{P}rv$ -local state D . Consequently, FL-DC integration with classic PoX faces the same challenges as LDP-DC.

B. ARM TrustZone-M Security Extensions

ARM TrustZone-M is a hardware security extension that enables a trusted execution environment (TEE) in ARM Cortex-M MCUs commonly used in low-cost and energy-efficient IoT applications. TrustZone divides software states into two isolated worlds: *Secure* and *Non-Secure*. The Non-Secure world contains and executes (untrusted) application software while security-critical (trusted) software is stored and runs in the Secure world.

In particular, we leverage two security properties of ARM TrustZone-M in this work:

- **Hardware-enforced World Isolation.** TrustZone-M ensures complete isolation of these worlds by implementing several hardware controls (i.e., SAU/IDAU) to enforce access control to hardware resources (e.g., program and data memory, peripherals) for these two worlds. With isolation in place, TrustZone ensures the Non-Secure World is unable to access any code and data located in the Secure World. This assures that the Secure world remains secure even if an adversary can fully modify or compromise the Non-Secure world software state.
- **Controlled Invocation.** In TrustZone-M, the only legal way for the Non-Secure world to access a function inside the Secure World is by making a call to predefined entry points. These entry points are located in a designated area within the Secure World, known as the Non-Secure Callable (NSC) region. As a part of the Secure World, the NSC region cannot be tampered with by the Non-Secure World's software. As a result, this mechanism combined with TrustZone-M secure context switching enables controlled invocation of the Secure World functions, preventing attacks that aim to compromise secure functions by executing them partially, i.e., by jumping into or exiting from the middle of the function.

III. SYSTEM MODEL AND ASSUMPTIONS

A. Network and Usage Model

We consider an IoT setting as shown earlier in Fig. 1, consisting of two entity types: one $\mathcal{Vrf}$ and multiple $\mathcal{Prv}$ -s. $\mathcal{Prv}$ is a resource-constrained sensor deployed in a physical space of interest, e.g., a smart home, office, or factory. $\mathcal{Vrf}$ is a remote service provider that orchestrates these $\mathcal{Prv}$ -s. As an example, $\mathcal{Prv}$ -s could be smart light bulbs that are used in many smart homes and can be controlled by the end-user through Philips $\mathcal{Vrf}$'s application services; or Samsung could act as $\mathcal{Vrf}$ that provides a service for the end-user to command all SmartThings-compatible devices.

Besides offering this service, $\mathcal{Vrf}$ wishes to collect sensor data generated by $\mathcal{Prv}$ -s to further improve the service performance or enhance the user experience. In this work, $\mathcal{Vrf}$ has the option to employ LDP-DC or FL-DC as its preferred data collection scheme, depending on the desired outcome and privacy considerations. As poisoning attacks could sabotage the collection outcome, $\mathcal{Vrf}$ also aims to detect and prevent such attacks to safeguard the authenticity of the outcomes.

B. Adversary Model

We consider an $\mathcal{Adv}$ who can modify/compromise $\mathcal{Prv}$'s application software at will. Once compromised, $\mathcal{Adv}$ can access, modify, or erase any code or data in $\mathcal{Prv}$ unless explicitly protected by hardware-enforced access control rules. Consequently, $\mathcal{Adv}$ may use this ability to perform poisoning attacks by corrupting a sensor function $\mathcal{F}$ or its execution to spoof arbitrary results sent to $\mathcal{Vrf}$. In the context of FL, in addition to data poisoning, $\mathcal{Adv}$ may use this capability to launch *model* poisoning attacks that aim to compromise the global machine learning model by introducing malicious local models (i.e., gradient updates) to $\mathcal{Vrf}$. Invasive hardware-based/physical attacks (e.g., fault-injection attacks or physical hardware manipulation) are out of scope in this work, as they require orthogonal tamper-proofing techniques [58].

We also assume $\mathcal{Adv}$ has full control over the communication channel between $\mathcal{Prv}$ and $\mathcal{Vrf}$. They may perform any network-based attacks, e.g., reading, modifying, replaying, or dropping any message sent from/to $\mathcal{Prv}$.

Further, we consider $\mathcal{Adv}$ to be adaptive [67], i.e., it is aware of the algorithm and all the specifics of the protocol executed between $\mathcal{Vrf}$ and $\mathcal{Prv}$. As a result, $\mathcal{Adv}$ is allowed to modify its attack strategy by modifying $\mathcal{Prv}$'s software state (except for hardware-enforced protections) and the communication between $\mathcal{Vrf}$ and $\mathcal{Prv}$ to attempt to circumvent the proposed defense.

Finally, in line with the standard LDP and FL $\mathcal{Adv}$ models, we also consider the possibility of malicious $\mathcal{Vrf}$. In the latter, $\mathcal{Adv}$'s goal is to learn sensitive data on $\mathcal{Prv}$ while executing the protocol.

C. Device Model

$\mathcal{Prv}$ -s are small embedded/IoT devices equipped with TrustZone-M, e.g., ARM embedded devices running on Cortex-M23/33 MCUs, which are optimized for low-cost and energy efficiency. Following the PoX assumption [13], we assume that the function whose execution is being proven ($\mathcal{F}$, i.e., the code implementing the data collection task according to the underlying scheme) is correct and contains no implementation bug that can lead to run-time exploits within itself. In practice, $\mathcal{Vrf}$ can employ various pre-deployment vulnerability detection techniques to fulfill this requirement [10].

We also adhere to standard TEE-based security assumptions, i.e., we assume the small TCB implementing the PoSX RoT located inside TrustZone's Secure World is trusted and TrustZone hardware modules are implemented correctly such that $\mathcal{Adv}$ cannot modify this RoT implementation or violate any security guarantees implemented by the Secure World-resident code. The latter implies the existence of secure persistent storage, exclusively accessible by the Secure World and unmodifiable when the device is offline. This storage is used to store our TCB along with a counter-based challenge c and two cryptographic keys: $sk_{\mathcal{Prv}}$ and $pk_{\mathcal{Vrf}}$, where $sk_{\mathcal{Prv}}$ corresponds to $\mathcal{Prv}$ private key, whose public counterpart $pk_{\mathcal{Prv}}$ is known to $\mathcal{Vrf}$. Similarly, $pk_{\mathcal{Vrf}}$ denotes $\mathcal{Vrf}$ public key with its private counterpart $sk_{\mathcal{Prv}}$ securely managed by $\mathcal{Vrf}$. In TrustZone-M implementations, secure persistent memory is

supported by a standard secure boot architecture [3] with the physical memory storing cryptographic keys being physically inaccessible through I/O interfaces (e.g., USB/J-TAG, etc.).

Finally, we assume these keys are correctly distributed to $\mathcal{Vrf}$ and $\mathcal{Prv}$ out-of-band, e.g., by physically fusing these keys on $\mathcal{Prv}$ during manufacture time or using any key-provisioning mechanism [43] after $\mathcal{Prv}$ deployment.

IV. PoSX AND ASSOCIATED DEFINITIONS

To define PoSX security goal, we start by revisiting classic PoX guarantees and their limitations. Then we formulate auxiliary notions that address each limitation and in conjunction imply PoSX end-to-end goal.

Definition 1 (PoX Security [13]). Let $\mathcal{F}$ represent an arbitrary software function (code) execution of which is requested by $\mathcal{Vrf}$ on $\mathcal{Prv}$, producing output $\mathcal{O}$. A protocol is considered PoX-secure if and only if the protocol outputting $\top$ implies:

- (i) $\mathcal{F}$ code (as defined by $\mathcal{Vrf}$) executes atomically and completely between $\mathcal{Vrf}$ sending a request and receiving the response, and
- (ii) $\mathcal{O}$ is a direct outcome of this execution of $\mathcal{F}()$.

Definition 1 states the classic PoX security notion, as described in [13]. A violation of conditions (i) or (ii) in Definition 1 must be detectable by $\mathcal{Vrf}$, resulting in a protocol abort (i.e., by outputting $\perp$). As discussed in [13], this notion can be used to construct “sensors that cannot lie” irrespective of compromised software states.

However, it only supports self-contained IoT applications that are independent of $\mathcal{Vrf}$ -defined inputs or pre-computed states due to two limitations:

L1 – Lack of input validation. Definition 1 considers *inputless* $\mathcal{F}$ functions. This is because classic PoX does not support verification of the origin and integrity of inputs received by $\mathcal{Prv}$. This limitation is significant for applications where $\mathcal{Vrf}$ must provide input $\mathcal{I}$ as part of $\mathcal{F}$ execution on $\mathcal{Prv}$ (e.g., FL/LDP cases discussed in Section II-A).

L2 – No state preservation across PoX instances. Definition 1 only supports PoX of stateless $\mathcal{F}$ functions. In other words, $\mathcal{F}$ must rely solely on data generated/acquired within its current execution instance and must not depend on $\mathcal{Prv}$ states (denoted $\mathcal{S}$) produced by prior executions. Similar to **L1** case, a PoX protocol satisfying Definition 1 provides no guarantee or validation to the correct use of some pre-existent/expected state $\mathcal{S}$ in $\mathcal{Prv}$. Thus, attacks that tamper with $\mathcal{S}$ in between subsequent PoX instances may result in illegal alteration of the end result $\mathcal{O}$.

Remark: when $\mathcal{S}$ contains only public information, **L2** can be obviated by **L1** by making $\mathcal{S}$ a part of the authenticated output of a PoX instance and used as a $\mathcal{Vrf}$ -defined input to $\mathcal{F}$ in a subsequent PoX instance. However, when $\mathcal{S}$ must remain hidden from $\mathcal{Vrf}$ (the case of our target applications – recall Section II-A), consistency of $\mathcal{S}$ must be ensured locally at $\mathcal{Prv}$, making **L1** and **L2** independent challenges.

To address these limitations systematically, we introduce two new PoX-related security notions. As these notions may be of independent interest, we first present them separately and finally compose them into an end-to-end PoSX goal.

Definition 2 (IV-PoX Security). Let $\mathcal{F}$ represent an arbitrary software function (code) and $\mathcal{I}$ represent an input, both defined by $\mathcal{Vrf}$. Let $\mathcal{O}$ represent the output and σ denote a proof of $\mathcal{F}(\mathcal{I})$ execution produced by $\mathcal{Prv}$. A protocol is considered IV-PoX-secure if and only if the protocol outputting $\top$ implies:

- (i) $\mathcal{F}$ executes with input arguments $\mathcal{I}$, atomically and completely between $\mathcal{Vrf}$ sending the request and receiving σ , and
- (ii) $\mathcal{O}$ is a direct outcome of this $\mathcal{F}(\mathcal{I})$ execution

To overcome **L1**, we present the notion of input-validating PoX (or IV-PoX), as shown in Definition 2. In IV-PoX, $\mathcal{Vrf}$ aims to execute $\mathcal{F}$ with its own provided input $\mathcal{I}$. Thus, the proof generated by $\mathcal{Prv}$, σ , must not only validate atomic and complete execution of $\mathcal{F}$ but also that it was invoked with the correct input requested by $\mathcal{Vrf}$ (as captured in condition (i) of Definition 2). Thus, an IV-PoX protocol must ensure $\mathcal{O}$ authenticity concerning $\mathcal{I}$, i.e., $\mathcal{O}$ is generated by executing $\mathcal{F}$ correctly using the expected input $\mathcal{I}$.

Definition 3 (SP-PoX Security). Let $\mathcal{F}$ represent an arbitrary software function (code) requested by $\mathcal{Vrf}$ to run on $\mathcal{Prv}$ with state $\mathcal{S}$, where $\mathcal{S}$ was produced by some prior PoX on $\mathcal{Prv}$ but is oblivious to $\mathcal{Vrf}$. Let $\mathcal{O}$ represent the output and σ denote a proof of $\mathcal{F}$ execution produced by $\mathcal{Prv}$. A protocol is considered SP-PoX-secure if and only if the protocol outputting $\top$ implies:

- (i) $\mathcal{F}$ executes atomically and completely between $\mathcal{Vrf}$ sending the request and receiving σ with $\mathcal{Prv}$ state corresponding to $\mathcal{S}$ when $\mathcal{F}$ execution starts (denote this execution by $\mathcal{F}_{\mathcal{S}}()$), and
- (ii) $\mathcal{O}$ is a direct outcome of this $\mathcal{F}_{\mathcal{S}}()$ execution, and
- (iii) $\mathcal{Vrf}$ cannot infer the value of $\mathcal{S}$ beyond what is revealed by $\mathcal{O}$, and
- (iv) $\mathcal{S}$ was not modified between the current $\mathcal{F}_{\mathcal{S}}()$ execution and the prior $\mathcal{Vrf}$ -authorized PoX.

To address **L2**, we specify the State-Preserving PoX (SP-PoX) notion in Definition 3. Similar to IV-PoX security, the SP-PoX notion specifies the first two conditions to ensure that, in addition to atomic $\mathcal{F}$ execution, σ also conveys two critical aspects: (1) correct use of $\mathcal{S}$ during $\mathcal{F}$ execution and (2) dependence of $\mathcal{O}$ on $\mathcal{F}$ and $\mathcal{S}$. In addition, it requires $\mathcal{S}$ privacy vis-a-vis $\mathcal{Vrf}$ and prohibits $\mathcal{S}$ modification in between subsequent PoX instances.

Finally, Definition 4 combines IV-PoX and SP-PoX to state the goal of PoSX-Security.

Definition 4 (PoSX Security). A scheme is PoSX-secure if and only if it satisfies both IV-PoX (Def. 2) and SP-PoX (Def. 3) Security.

V. SLAPP: REALIZING PoSX WITH TRUSTZONE-M

A. Overview of SLAPP Workflow

Building on TrustZone-M hardware-enforced world isolation (recall Section II-B), our approach is to implement SLAPP’s RoT in and execute it from the Secure World. Meanwhile, normal applications are untrusted (hereby referred to as untrusted software) and reside in the Non-Secure World.

SLAPP implements three Secure World functions: Execute, CheckState, and SetState. Execute serves as the main call to execute $\mathcal{F}_{\mathcal{S}}(\mathcal{I})$ on $\mathcal{Prv}$ and compute a proof of this stateful execution. CheckState and SetState are used to authenticate state $\mathcal{S}$ used in Execute.

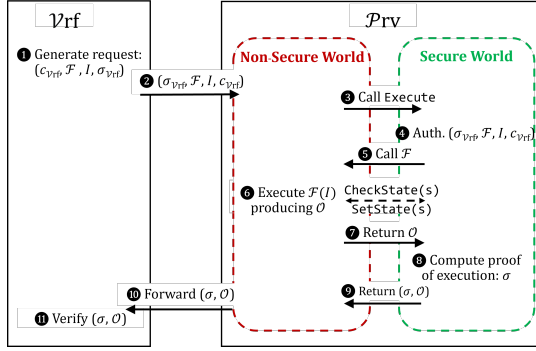


Fig. 2: Overview of SLAPP workflow

SLAPP workflow is depicted in Fig. 2. At a high level, it enforces the following operation sequence upon receiving a PoSX request from $\mathcal{Vrf}$:

- Upon being called by the Non-Secure World, Execute authenticates $\mathcal{Vrf}$ request in ④.
- Following successful authentication, it starts executing $\mathcal{F}_S(\mathcal{I})$ atomically in Non-Secure World, ⑤.
- Before accessing $\mathcal{S}$ in ⑥, $\mathcal{F}_S(\mathcal{I})$ must call CheckState in the Secure World. CheckState takes one input argument, representing the current $\mathcal{S}$ value. Its task is to authenticate this value by matching it against the latest benign $\mathcal{S}$ value from a prior execution stored within the Secure World.
- Similarly, after $\mathcal{S}$ is modified in ⑥, execution is trapped into the Secure World via the SetState function. SetState accepts one input argument: the new $\mathcal{S}$ value. It is responsible for committing and securely maintaining the latest benign $\mathcal{S}$ value in the Secure World.
- Once $\mathcal{F}_S(\mathcal{I})$ execution completes, yielding output $\mathcal{O}$, the control returns to Execute in ⑦. Then, Execute computes σ indicating an authenticated proof of this execution in ⑧ and returns $(\mathcal{O}, \sigma)$ to the Non-Secure World in ⑨, which in turn forwards them to $\mathcal{Vrf}$.

With $(\mathcal{O}, \sigma)$, $\mathcal{Vrf}$ can determine if $\mathcal{F}_S(\mathcal{I})$ was executed in $\mathcal{Prv}$'s Non-Secure World, if $\mathcal{O}$ is an authentic result of this execution, and if $\mathcal{S}$ was preserved since the prior execution. Importantly, SLAPP assures that σ is not computable unless the aforementioned operation sequence is observed.

B. SLAPP in Detail

Protocol 1 details the $\mathcal{Vrf} \leftrightarrow \mathcal{Prv}$ interaction in SLAPP.

Phase 1: Request Generation

An instance of SLAPP protocol starts when $\mathcal{Vrf}$ generates a PoSX request, in Step 1, comprising: (i) an identifier $\mathcal{F}$ for which function to execute; (ii) input arguments $\mathcal{I}$; and (iii) a monotonically-increasing counter-based challenge $c_{\mathcal{Vrf}}$. This request instructs $\mathcal{Prv}$ to execute $\mathcal{F}_S(\mathcal{I})$ in the Non-Secure World on $\mathcal{Prv}$ and return the result to $\mathcal{Vrf}$ along with proof of this expected execution. Following this, $\mathcal{Vrf}$ signs this request (via Sign function), producing token $\sigma_{\mathcal{Vrf}}$, and attaches this token to the request before sending it to $\mathcal{Prv}$ in Step 2.

Phase 2: Preparation for $\mathcal{F}_S(\mathcal{I})$ Execution

In Step 3, untrusted software in $\mathcal{Prv}$ must invoke Execute passing the PoSX request as a parameter. In step 4, Execute

(i) checks whether the request counter $c_{\mathcal{Vrf}}$ is larger than a local counter c maintained by the Secure World to ensure freshness of the PoSX request and prevent replayed attacks; (ii) verifies $\sigma_{\mathcal{Vrf}}$ to confirm authenticity of the request; and (iii) checks that no other SLAPP instance is active by examining $exec$ flag maintained in the secure world. If any of the checks fail, the process is aborted which implies the inability to produce the end proof σ . The same applies if Execute is never called.

Protocol 1: SLAPP Protocol

Verifier ($\mathcal{Vrf}$)

- (1) Generate an authenticated PoSX request:

$$\sigma_{\mathcal{Vrf}} \leftarrow \text{Sign}(\text{sk}_{\mathcal{Vrf}}, H(\mathcal{F}, \mathcal{I}, c_{\mathcal{Vrf}}))$$

- (2) Send $(\sigma_{\mathcal{Vrf}}, \mathcal{F}, \mathcal{I}, c_{\mathcal{Vrf}})$ to $\mathcal{Prv}$

Prover ($\mathcal{Prv}$)

In Non-Secure World:

- (3) Call Execute in Secure World with the received request

In Execute function, Secure World:

- (4) Authenticate the request and abort if $r = \perp$ or $exec = \top$:

$$r \leftarrow (c_{\mathcal{Vrf}} > c) \wedge \text{Verify}(\text{pk}_{\mathcal{Vrf}}, \sigma_{\mathcal{Vrf}}, \mathcal{F}, \mathcal{I}, c_{\mathcal{Vrf}})$$

- (5) Update counter: $c \leftarrow c_{\mathcal{Vrf}}$ and initialize: $exec \leftarrow \top$, $stateUsed \leftarrow \perp$, $stateChecked \leftarrow \perp$
- (6) Disable interrupts and measure $PMEM$ and $\mathcal{Vrf}$ request:

$$h \leftarrow H(PMEM, \mathcal{F}, \mathcal{I}, c_{\mathcal{Vrf}})$$

- (7) Call $\mathcal{F}_S(\mathcal{I})$ in Non-Secure World

In $\mathcal{F}$ function, Non-Secure World:

- (8) Run $\mathcal{F}_S(\mathcal{I})$. Pass control to the Secure World when $\text{CheckState}(s)$ is called

In CheckState function, Secure World:

- (9) Perform an integrity check on s , store the result to p and return to Non-Secure World: $stateChecked \leftarrow (H(s) \stackrel{?}{=} S_{sec})$. Also, set $stateUsed \leftarrow \top$.

In $\mathcal{F}$ function, Non-Secure World:

- (10) Continue with $\mathcal{F}_S(\mathcal{I})$ execution. Pass control to the Secure World when $\text{SetState}(s)$ is called

In SetState function, Secure World:

- (11) Securely set S_{sec} based on input s and return to Non-Secure World:

$$S_{sec} \leftarrow H(s) \text{ if } (stateChecked \wedge exec)$$

In $\mathcal{F}$ function, Non-Secure World:

- (12) Continue with $\mathcal{F}_S(\mathcal{I})$ until the execution is completed, producing output $\mathcal{O}$, and then return to its caller, Execute, with $\mathcal{O}$

In Execute function, Secure World:

- (13) Abort if $(exec \wedge stateUsed \wedge \neg stateChecked)$. Otherwise, include $\mathcal{O}$ to the measurement and compute the proof:

$$\sigma \leftarrow \text{Sign}(\text{sk}_{\mathcal{Prv}}, H(h, \mathcal{O}))$$

- (14) Reset all flags: $exec \leftarrow \perp$, $stateChecked \leftarrow \perp$, $stateUsed \leftarrow \perp$
- (15) Enable interrupts and return with $(\mathcal{O}, \sigma)$

In Non-Secure World:

- (16) Forward $(\mathcal{O}, \sigma)$ to $\mathcal{Vrf}$

Verifier ($\mathcal{Vrf}$)

- (17) Increment $c_{\mathcal{Vrf}}$ and validate σ by:

$$r \leftarrow \text{ValidatePoSX}(\text{pk}_{\mathcal{Prv}}, \sigma, PMEM', \mathcal{F}, \mathcal{I}, c_{\mathcal{Vrf}}, \mathcal{O})$$

The protocol outputs r .

If all checks succeed, Execute updates the local counter with $c_{\mathcal{Vrf}}$ and initializes three Secure World flags in Step 5:

- $exec$ is set to $\top$, indicating an active SLAPP instance.

- *stateChecked* is set to $\perp$, indicating the status of S authenticity during $\mathcal{F}$ execution.
- *stateUsed* is set to $\perp$, indicating the status of S access during $\mathcal{F}$ execution.

In Step 6, Execute prepares $\mathcal{P}_{rv}$ for the upcoming execution of $\mathcal{F}$ by disabling all interrupts and taking a snapshot (h) as a hash digest reflecting the states of the Non-Secure World's binary code in program memory ($PMEM$) and the parameters received in the PoSX request. It then calls $\mathcal{F}$ on input $\mathcal{I}$ in the Non-Secure World (Step 7).

To leverage SLAPP, the implementation of $\mathcal{F}$ must:

- B1:** At the start of its execution, validate relevant state S authenticity by calling CheckState.
- B2:** At the end of its execution, commit the latest S value to the Secure World by invoking SetState.
- B3:** Throughout its execution, $\mathcal{F}$ must never enable interrupts.

Phase 3: $\mathcal{F}_S(\mathcal{I})$ Execution

From **B1**, it follows that a call to $\mathcal{F}$, in Step 8, triggers CheckState of current S . In Step 9, CheckState, executed in the Secure World, sets *stateUsed* to $\top$ and proceeds to verify authenticity of the current S value, s , by comparing s with the latest benign S value, S_{sec} , stored in the Secure World. Only if they match, CheckState ascertains s authenticity setting *stateChecked* to $\top$. Execution of $\mathcal{F}$ is then resumed. From **B2**, SetState is invoked at the end of $\mathcal{F}$ execution to update S_{sec} with the new state s if and only if (*stateChecked* = *exec* = $\top$). To optimize storage, especially when $|s|$ is large, SetState can update S_{sec} with a hash of s , as shown in Step 11. This storage optimization comes with the expense of additional runtime overhead for hash computations in CheckState and SetState. We discuss this time-space trade-off further in Section VII.

$\mathcal{F}$ execution completes producing the output $\mathcal{O}$ and handing the control to Execute with $\mathcal{O}$ as an input in Step 12.

Phase 4: Proof Generation

Execute examines *exec*, *stateUsed*, and *stateChecked* flags to determine the occurrence of the CheckState $\rightarrow$ SetState sequence. If this sequence is maintained during $\mathcal{F}$ execution, Execute proceeds to compute the proof σ in Step 13 by signing h and $\mathcal{O}$ using $\mathcal{P}_{rv}$'s private key $sk_{\mathcal{P}_{rv}}$. Execute resets all Secure World flags before returning to the Non-Secure World with $\mathcal{O}$ and σ , in Step 15. They are then transmitted to $\mathcal{V}_{rf}$, in Step 16.

Phase 5: Proof Validation

Upon receiving $\mathcal{O}$ and σ , $\mathcal{V}_{rf}$ increments $c_{\mathcal{V}_{rf}}$ and performs the PoSX verification in Step 17 by:

- Checking validity of σ . As $\mathcal{V}_{rf}$ possesses the expected binary of $\mathcal{P}_{rv}$'s Non-Secure World, $PMEM'$, this verification involves checking σ against $PMEM'$, $\mathcal{F}$, $\mathcal{I}$, $c_{\mathcal{V}_{rf}}$ and $\mathcal{O}$ using $\mathcal{P}_{rv}$'s public key $pk_{\mathcal{P}_{rv}}$, i.e.:

$$\text{Verify}(pk_{\mathcal{P}_{rv}}, \sigma, PMEM', \mathcal{F}, \mathcal{I}, \mathcal{O}, c_{\mathcal{V}_{rf}}) \stackrel{?}{=} \top$$

- Inspect $\mathcal{F}$ binary to ensure that it adheres to the expected behaviors: **B1**, **B2** and **B3**.

Finally, SLAPP protocol is considered successful and thus outputs $\top$ if it passes both checks; it aborts with $\perp$ otherwise.

VI. PoSX SECURITY ANALYSIS

Our security argument is based on the following properties:

P1 - Request Verification. In SLAPP, $\mathcal{P}_{rv}$'s TCB always verifies freshness and authenticity of a PoSX request before executing $\mathcal{F}$ and generating the proof σ . This prevents $\mathcal{A}_{dv}$ from exploiting forged or replayed requests to manipulate the protocol outcome. See step 4 of Protocol 1.

P2 - Input Validation. A valid σ serves as authentication for the correct usage of $\mathcal{I}$ during $\mathcal{F}$ execution. This prevents $\mathcal{A}_{dv}$ from feeding malicious input to $\mathcal{F}$ while still succeeding in the SLAPP protocol. Since $\mathcal{I}$ is included in a PoSX request, this is implied by **P1** and the fact that $\mathcal{I}$ is directly used by SLAPP to invoke $\mathcal{F}$ in step 7 of Protocol 1.

P3 - State Privacy. SLAPP protects privacy of S from $\mathcal{V}_{rf}$ since the only information $\mathcal{V}_{rf}$ receives are $\mathcal{O}$ and σ . σ is not a function of S ; thus it leaks nothing about S to $\mathcal{V}_{rf}$. Thus, SLAPP incurs no leakage other than $\mathcal{O}$ itself.

P4 - State Authenticity. In SLAPP, the successful completion of a protocol instance guarantees the S value, stored in the Non-Secure World, is authentic, i.e., it can only be modified by $\mathcal{V}_{rf}$ -approved software during a protocol instance and remains unchanged between consecutive instances. This assurance comes from two observations:

First, S_{sec} always corresponds to the latest benign S value because: (1) S_{sec} cannot be updated outside a protocol instance due to the check of *exec* flag in Step 11; and (2) S_{sec} cannot be influenced by forged or replayed PoSX requests since the request is always authenticated in Step 4 before *exec* can be set in Step 5.

Second, any unauthorized modification to S outside a protocol instance is detected in the subsequent instance by CheckState due to a mismatch between the S value and S_{sec} . As a successful SLAPP protocol implies a successful check from CheckState, S must be equal to S_{sec} , and, according to the first observation, must contain the latest authentic S value. Moreover, $\mathcal{A}_{dv}$ may attempt to tamper with S while $\mathcal{F}$ is executing. However, doing so requires modification to $PMEM$, which would result in a mismatch with $PMEM'$ during the proof validation in Step 17.

P5 - Atomic Execution. $\mathcal{F}$ execution must occur atomically; otherwise, the protocol must fail. This is required to prevent $\mathcal{A}_{dv}$ from interrupting $\mathcal{F}$ execution to tamper with its data and execution flows, influencing the outcome. Step 6 realizes this requirement by disabling all interrupts before $\mathcal{F}$ invocation. We explain how this requirement can be relaxed in Section VII.

P6 - Output Authenticity. A successful SLAPP protocol indicates to $\mathcal{V}_{rf}$ that $\mathcal{O}$ is authentic and generated by executing $\mathcal{F}$ atomically with the $\mathcal{V}_{rf}$ -specified $\mathcal{I}$ and the correct state S . SLAPP satisfies this property since **P1** guarantees that $\mathcal{F}$ is always invoked with authentic $\mathcal{I}$ and **P5** enforces $\mathcal{F}$ to execute without interruptions before immediately returning to Secure World with $\mathcal{O}$. This leaves no opportunities for untrusted software to interrupt $\mathcal{F}$ execution to tamper with $\mathcal{O}$. $\mathcal{A}_{dv}$ attempt to change $\mathcal{O}$ via S are prevented by **P4**.

Security Argument. We show that SLAPP satisfies both Definition 2 and Definition 3 implying adherence to Definition 4. Per Definition 2, IV-PoX security requires PoX assurance for stateless $\mathcal{F}$ functions that run with input arguments, i.e., when $\mathcal{S} = \phi$ and $\mathcal{I} \neq \phi$. In this scenario, **P2** and **P5** ensure the atomic execution of $\mathcal{F}$ with the authentic input $\mathcal{I}$ provided by $\mathcal{Vrf}$, satisfying condition (i). **P6** guarantees that $\mathcal{O}$ is generated as a result of $\mathcal{F}(\mathcal{I})$ execution, fulfilling condition (ii). With both conditions met, SLAPP achieves IV-PoX security. Meanwhile, SV-PoX security in Definition 3 requires secure PoX of a stateful $\mathcal{F}$ executed without input arguments. Similar to the previous argument, **P5** and **P6** directly address conditions (i) and (ii) of SV-PoX security even when $\mathcal{I} = \phi$. As **P3** ensures that SLAPP leaks nothing about $\mathcal{S}$ besides its intended execution output, it fulfills condition (iii). Also, **P4** guarantees that $\mathcal{S}$ cannot be modified except by a fresh instance of SLAPP protocol, which satisfies condition (iv). Meeting all conditions in Definition 3, SLAPP is also SP-PoX-secure. Lastly, PoSX security (per Definition 4) follows directly from simultaneous adherence to IV-PoX and SP-PoX security. $\square$

Remark. SLAPP maintains **P1-P6** even in the presence of an adaptive $\mathcal{Adv}$; this implies that irrespective of $\mathcal{Adv}$ actions or knowledge of SLAPP (assuming no invasive hardware attacks), none of these properties can be compromised.

VII. SLAPP EXTENSIONS AND VARIATIONS

Cryptographic Choices. Although Protocol 1 uses public-key cryptography, it can seamlessly transition to symmetric cryptography by simply substituting public-key operations (i.e., Sign and Verify) with MAC operations. If quantum threats are in scope, SLAPP can be similarly adjusted to support a post-quantum signature scheme. We demonstrate this versatility by implementing our prototype (Section IX) using three distinct cryptographic choices (public-key, symmetric, and post-quantum cryptography).

Space-Time Trade-Off. Section V-B discussed choices for managing $\mathcal{S}_{sec}$ in the Secure World: (1) storing the entire $\mathcal{S}$ value; or (2) maintaining a hash of $\mathcal{S}$ value. The first prioritizes runtime efficiency since it requires no run-time hash computation while the second conserves storage in the Secure World by condensing the potentially large $\mathcal{S}$ into a fixed-size digest. A third option, that eliminates storage in the Secure World, is to have SetState compute a MAC of $\mathcal{S}$ (instead of a hash) and pass it back to the Non-Secure World for storage. Subsequently, the Secure World can authenticate $\mathcal{S}$ received from the Normal World based on the MAC, yielding equivalent security guarantees as the previous two approaches. By default, SLAPP prototype adopts the second design choice, striking a balance between space and time overhead.

Multiple Stateful Functions. Our description of SLAPP assumes that $\mathcal{Vrf}$ intends to obtain one PoSX per $\mathcal{Prv}$ at a time. Nonetheless, it can be extended to accommodate simultaneous PoSX-s by maintaining a map between multiple $\mathcal{S}_{sec}$ -s and each ongoing PoSX, in the Secure World. To that end, CheckState and SetState should be extended to assign $\mathcal{S}_{sec} = \text{map}[id]$ for a given function identifier id . Also, in

the last step of the protocol, $\mathcal{Vrf}$ must additionally inspect $\mathcal{F}$ binary to ensure that it correctly calls CheckState and SetState with the correct function identifier.

Relaxing Atomicity Requirement. SLAPP security mandates atomicity (uninterruptability) during $\mathcal{F}$ execution. This requirement may clash with real-time needs on $\mathcal{Prv}$, potentially preventing time-sensitive tasks from completing while SLAPP is running. Recent studies [8], [48] propose techniques to relax this atomicity requirement in classic PoX and related schemes. These can also be adopted in SLAPP as follows: rather than completely disabling interrupts, the Secure World “locks” (i.e., making them read-only) $\mathcal{PMEM}$, data currently in use by the PoSX task, and the Interrupt Vector Table (IVT). It also includes IVT in the snapshot h before invoking $\mathcal{F}$. As a consequence, the PoSX context is protected across interrupts. Once $\mathcal{F}$ completes, respective memory can be unlocked.

VIII. FROM PoSX TO POISONING PREVENTION

We now discuss how SLAPP can be leveraged to detect poisoning attacks in LDP-DC and FL-DC.

A. Poisoning-free LDP.

LDP-DC⁺ relies on SLAPP to ensure the correct execution of the following steps:

- 1) **Setup.** Run SLAPP protocol to obtain a PoSX of $\text{Init-state}_B(\phi)$, which executes the function Init-state without any input using the state variable B (for $\mathcal{PRR}$ mapping). This execution initializes B to an empty list on $\mathcal{Prv}$. Abort if the protocol outputs $\perp$.
- 2) **Collect.** Run SLAPP protocol to obtain a PoSX of $\text{LDP-DC}_B(\mathcal{F}, \mathcal{I}, f, p, q)$ as specified in Algorithm 1. On state B , this execution performs a sensor reading $\mathcal{F}(\mathcal{I})$, perturbs the reading result using the LDP-based mechanism with the parameter values f , p , and q , and returns the noisy output $\mathcal{O}$ to $\mathcal{Vrf}$. Repeat this step if $\mathcal{Vrf}$ wants to collect more readings.

SLAPP in **Setup** step ensures that the state variable B is initialized to an empty value. At a later time, **Collect** can be executed, where SLAPP protocol gives assurance to $\mathcal{Vrf}$ that: (1) $\mathcal{O}$ is genuine, originating from a timely execution of the sensor function, and correctly privatized by the underlying LDP mechanism, (2) B corresponded to the authentic value (e.g., empty at the first time of this step’s execution) right before and during the protocol execution, and (3) B is correctly updated as a result of the protocol execution. These prevent poisoning attacks because $\mathcal{Adv}$ can tamper with neither $\mathcal{O}$ (during **Collect**) nor B (during **Setup** or **Collect**).

B. Poisoning-free FL.

FL-DC⁺ follows a similar approach by using SLAPP to convey to $\mathcal{Vrf}$ the correct execution of:

- 1) **Setup.** Run SLAPP protocol to obtain a PoSX of $\text{Init-State}_D(\phi)$, which executes without input arguments and uses the local training dataset D as the underlying PoSX state. The execution sets D to an empty list on $\mathcal{Prv}$. Abort if it outputs $\perp$.

- 2) **Collect.** To perform a sensor reading $\mathcal{F}(\mathcal{I})$ and record the authentic result to the state D , run SLAPP to obtain a PoSX of Sense-Store $_D(\mathcal{F}, \mathcal{I})$; see Algorithm 2. Repeat this step to collect more sensor readings. Abort if the protocol outputs $\perp$.
- 3) **Train.** Run SLAPP protocol to obtain a PoSX of Train $_D(W, t, \alpha)$ as specified in Algorithm 2. This execution performs local training on the state D using the $\mathcal{Vrf}$ -requested global weights W and training parameters t and α ; it then outputs the locally trained model $\mathcal{O}$ to $\mathcal{Vrf}$.

Similar to LDP-DC⁺, **Setup** guarantees to $\mathcal{Vrf}$ that D starts empty. For **Collect**, SLAPP ensures that each record in D is produced as a result of executing the expected sensor function on $\mathcal{P}rv$. Finally, **Train** assures $\mathcal{Vrf}$ that the correct training function was applied to the authentic training data in D , leading to the received trained model $\mathcal{O}$. In the context of FL, $\mathcal{Adv}$ may attempt poisoning attacks in two ways: (1) model poisoning by directly tampering with the trained model $\mathcal{O}$ and (2) data poisoning by manipulating the dataset D used for training. Both are prevented by FL-DC⁺.

Remark. We emphasize that both LDP-DC⁺ and FL-DC⁺ maintain the privacy of their original counterparts. The only additional information besides $\mathcal{O}$ obtained by $\mathcal{Vrf}$ is σ , which is not a function of the underlying execution state. Also, as a system-level approach, these schemes offer a deterministic guarantee in discerning poisoned data from authentic data, i.e., achieving 100% true positive and true negative rates, irrespective of adaptive attacks. It also comes without any assumptions about the data distributions on $\mathcal{P}rv$.

IX. EVALUATION

A. Experimental Setup

Cryptographic Variants. As noted in Section VII, SLAPP is agnostic to underlying cryptographic primitives. To showcase this flexibility, we provide 3 implementation variants of SLAPP’s RoT: SLAPP_{SK}, SLAPP_{PK}, and SLAPP_{PQ}:

- 1) SLAPP_{SK} uses symmetric-key cryptography to implement Sign and Verify in Protocol 1 with an HMAC-SHA256 in line with prior PoX work [13].
- 2) SLAPP_{PK} relies on the public-key signature ECDSA NIST256p from micro-ECC library¹, which is commonly used in embedded settings [60], [45].
- 3) SLAPP_{PQ} implements Sign and Verify using the quantum-resistant public-key signature Sphincs+, a low-RAM version of Sphincs-sha2-128f². This choice is supported by prior research [49], [30] showing feasibility of Sphincs+ on Cortex-M devices.

Prototype. We prototype SLAPP variants on a NUCLEO-L552ZE-Q [61] development board, representing resource-constrained IoT devices. It features TrustZone-M on an ARM Cortex-M33 MCU @ 110MHz clock, with 512KB of FLASH (of which we assign 256KB to store the Non-secure World’s PMEM) and 256KB of RAM. To accurately isolate SLAPP

TABLE I: Binary size (in KB) of TCB.

Variants	Baseline	SLAPP	Overhead
Symmetric-key	17.0	17.5	2.9%
Public-key	34.5	35.0	1.4%
Post-quantum	24.0	24.5	2.0%

overheads, we implement a simple stateful $\mathcal{F}$ function on $\mathcal{P}rv$ that takes input from a GPIO port specified from a PoSX request, performs a sensor reading on that port, and outputs an accumulative sum of all readings over time to $\mathcal{Vrf}$. $\mathcal{Vrf}$ is deployed as a commodity desktop equipped with an Intel i5-9300H CPU @ 2.4GHz. $\mathcal{Vrf}$ and $\mathcal{P}rv$ are connected via serial communication. All prototypes are open-sourced and publicly available at [57].

B. Baseline.

For comparison, we consider an alternative naive baseline approach in which all functions to prove execution $\mathcal{F}$ are included as part of TCB in the Secure World. To perform PoSX, TCB receives and authenticates a request from the Non-Secure World, just like SLAPP. Unlike SLAPP, this approach executes $\mathcal{F}$ inside the Secure World. The result is signed (or MAC’ed) and forwarded to $\mathcal{Vrf}$, akin to SLAPP. As this baseline approach is also agnostic to the underlying cryptographic primitives, we refer to Baseline_{SK}, Baseline_{PK} and Baseline_{PQ} as the baseline approaches that utilize symmetric-key cryptography (HMAC-SHA256), public-key cryptography (ECDSA) and post-quantum cryptography (Sphincs+), respectively.

We note that this baseline faces several security and practical downsides. First, its TCB becomes bloated and dependent on multiple untrusted applications by including all $\mathcal{F}$ -s within the Secure World. This implies that a vulnerability in one of them can compromise all (i.e., violating the principle of least privilege). It also incurs Secure World’s additional storage for maintaining data of $\mathcal{F}$ execution. This reduces available RAM for normal applications in the Non-Secure World. Moreover, it makes the Secure World code (which should be immutable post-deployment – recall Section II-B) application-specific and thus requires rewriting/updating the Secure World every time to support new applications, which may necessitate cumbersome physical intervention.

C. Space Overhead

Code Size. SLAPP’s TCB was implemented in C and compiled using the -O3 optimization flag. Details of code size are shown in Table I. As SLAPP_{SK} relies on an inexpensive cryptographic primitive, it exhibits the smallest code size. Conversely, SLAPP_{PK} results in the largest binary size of 35.0KB while SLAPP_{PQ}’s binary is around 24.5KB. Compared to the baselines, SLAPP introduces a small code size overhead (0.5KB), corresponding to 1.4-2.9% across all variants.

For $\mathcal{F}$ instrumentation, SLAPP requires prepending a call to CheckState at the beginning of $\mathcal{F}$ and another call to SetState before $\mathcal{F}$ returns. This instrumentation results in only 2 additional lines of C code, enlarging $\mathcal{F}$ ’s binary (residing in Non-Secure World) by a fixed 22 bytes.

Memory Usage. Next, we estimate the peak amount of Secure World data allocated at runtime. This data consists

¹<https://github.com/kmackay/micro-ecc>

²<https://github.com/sphincs/low-ram-sphincsplus>

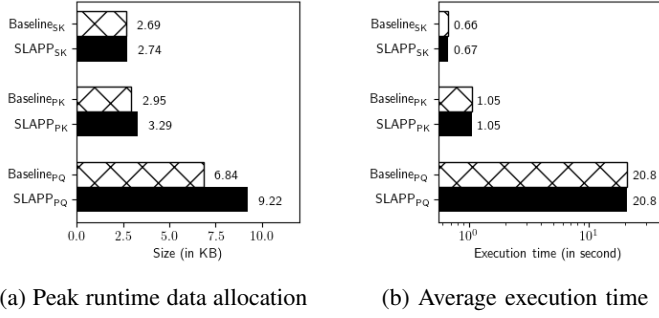


Fig. 3: Resource usage across all approaches

of all stack and static/global variables (our implementation utilizes no heap allocation). As shown in Fig. 3a, SLAPP_{SK} and SLAPP_{PK} uses roughly the same amount of data: 2.74 and 3.29KB. SLAPP_{PQ} , on the other hand, requires more than triple this amount: 9.22KB. Compared to the baselines, our SLAPP variants incur an additional memory usage ranging from 50 bytes for SLAPP_{SK} to 2.38KB for SLAPP_{PQ} . These correspond to $< 4\%$ of available RAM in the Secure World.

D. Time Overhead

As $\mathcal{Vrf}$ operates on a powerful back-end, its runtime within the SLAPP protocol is negligible compared to the execution time on $\mathcal{Prv}$. Thus, we focus on measuring $\mathcal{Prv}$'s execution time (i.e., the runtime time of executing Phases 3, 4, and 5 in Section V). Results are illustrated in Fig. 3b.

As expected, SLAPP_{SK} has the fastest runtime: 0.67s. In contrast, SLAPP_{PQ} utilizes expensive cryptography and thus incurs the longest execution time of $\approx 20\text{s}$. SLAPP_{PK} positions between these two, taking around 1 second.

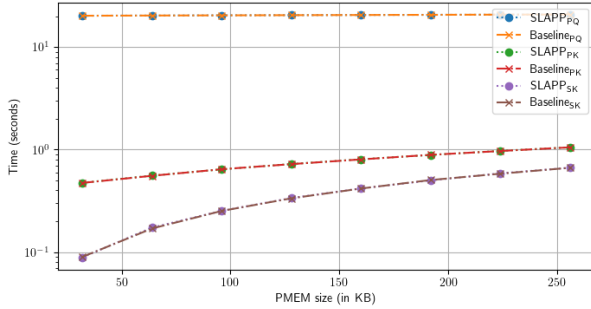


Fig. 4: Execution time with varying $PMEM$ size

Since SLAPP requires $\mathcal{Prv}$ to compute a snapshot of $PMEM$ as part of proof generation, we conducted an experiment to assess the impact of $PMEM$ size on $\mathcal{Prv}$'s execution time. In this experiment, we varied the $PMEM$ size from 32KB to 256KB. As depicted in Fig. 4, the time to execute this snapshot contributes significantly to the overall execution time when SLAPP_{SK} is used. It also shows a linear relationship between execution time and $PMEM$ size for SLAPP_{SK} and SLAPP_{PK} . However, for SLAPP_{PQ} , this effect is negligible; its runtime has minimal impact on the overall execution time regardless of $PMEM$ size.

From Figs. 3b and 4, we can conclude that all SLAPP variants incur a negligible execution time overhead, i.e., $< 1\%$, compared to their baseline counterparts. We next conduct end-to-end evaluation of SLAPP through two case studies.

E. Case Study 1: Local Differential Privacy

Description. In the first case study, we envision the integration of SLAPP within a smart city/grid system. The service provider aims to periodically collect energy consumption data from all smart meters located in individual households to calculate electric bills, forecast load, etc. Previous studies [40], [44] have demonstrated privacy risks by exposing raw energy to the service provider, e.g., with access to such data, the provider could potentially infer users' habits and behavior. LDP-DC (from Algorithm 1) can be employed to address this concern. The service provider is also motivated to use a poisoning-free version, LDP-DC⁺ as presented in Section VIII-A, to prevent poisoning attacks from potentially malicious edge devices.

We build a prototype of a smart meter ($\mathcal{Prv}$) based on NUCLEO-L552ZE-Q connecting to a PZEM-004T energy meter hardware module. $\mathcal{Prv}$'s Non-Secure World consists of LDP-DC software (Algorithm 1) and a driver responsible for retrieving energy data from the PZEM-004T hardware module. Meanwhile, the Secure World contains SLAPP_{SK} 's TCB. The service provider ($\mathcal{Vrf}$) runs on a commodity desktop and communicates with $\mathcal{Prv}$ over serial communication.

During normal (benign) operation, $\mathcal{Vrf}$ and $\mathcal{Prv}$ execute an instance of LDP-DC⁺ protocol. This instance begins with the **Setup** step, which requires PoSX of Init-State function to initialize a $\mathcal{Prv}$ -local state (PRR mapping) to zeroes. $\mathcal{Vrf}$ verified that $\mathcal{Prv}$ faithfully executes this step, completing with $\top$. Upon obtaining the successful output, $\mathcal{Vrf}$ proceeds to the **Collect**, which aims to collect authentic noisy energy data from $\mathcal{Prv}$. To achieve this, $\mathcal{Vrf}$ makes a PoSX request by configuring a function to prove execution to LDP-DC and properly selecting LDP input parameters (i.e., f , p and q) to meet the ϵ -LDP requirement, and sends an authenticated PoSX request to $\mathcal{Prv}$. To successfully respond to this PoSX request, $\mathcal{Prv}$ must execute LDP-DC correctly (with $\mathcal{Vrf}$ -defined inputs) and return the result (authentic noisy energy data) to $\mathcal{Vrf}$. If this step completes with $\top$, it ensures $\mathcal{Vrf}$ that the received energy data is not poisoned.

End-to-end Evaluation. Results for this case study are presented in Table II. SLAPP_{SK} takes around 17.4KB and 8.5KB of Secure and Non-Secure FLASH, respectively. $\text{Baseline}_{\text{SK}}$ would require placing the code implementing the LDP logics into Secure FLASH, enlarging its TCB by 5.8KB or 33.3%. Note that this number would further increase with complexity of functionality or if multiple different sensing functions need to be implemented by $\mathcal{Prv}$. This result substantiates SLAPP's design rationale: being application-agnostic significantly reduces the TCB size and provides flexibility. Regarding RAM usage, SLAPP_{SK} requires 4.7KB of RAM to execute the **Setup** and **Collect** steps, which is 0.2KB less than $\text{Baseline}_{\text{SK}}$, leading to 4.3% reduction in Secure RAM usage.

For both steps, the end-to-end execution time (from $\mathcal{Vrf}$ generating a request to $\mathcal{Vrf}$ verifying the response) is ap-

TABLE II: End-to-end results for Baseline_{SK} (shown in parentheses) and SLAPP_{SK}. Phases 1-5 are as defined in Section V-B.

Case Study	Protocol	Step	Average Execution Time (ms)					Total	RAM (KB)		FLASH (KB)	
			Phase 1	Phase 2	Phase 3	Phase 4	Phase 5		S	NS	S	NS
(1) Collect noisy energy readings	LDP-DC ⁺	Setup	8	660	7	1	11	687(678)	4.7(4.9)	2.7(1.2)	17.4(23.2)	8.5(7.4)
		Collect	6	660	9	1	8	684(677)	4.7(4.9)	2.7(1.2)		
(2) Train a model for load forecasting	FL-DC ⁺	Setup	13	652	8	1	8	682(678)	4.7(5.4)	3.3(1.5)	17.4(71.3)	57.2(7.5)
		Collect	12	654	8	1	7	682(682)	4.7(5.4)	3.3(1.5)		
		Train	16	651	3,067	2	20	3,756(3,759)	4.7(32.0)	32.0(1.5)		

proximately 0.69 seconds, corresponding to 1% increase over Baseline_{SK}. We also reported the execution time breakdown for all phases in Table II. Recall Section V-B for the definition of each phase. As Phase 2 requires a hash computation over the Non-Secure FLASH of 256KB, it dominates the end-to-end runtime, accounting for $\approx 96\%$ of the overall runtime. Phase 4 is the fastest, as it consists of only lightweight operations, i.e., setting flags and computing HMAC on a short message, without requiring any cross-world switching or network communication. Finally, the time taken by $\mathcal{V}rf$ (Phases 1 and 5) contributes with $< 3\%$ of the overall runtime.

We next consider a task of using LDP-DC⁺ to collect a variable number of noisy energy readings from 1 to 20. This emulates the case of continuous data collection to be performed over a longer period of time. For instance, $\mathcal{V}rf$ requests 1 reading every 3 minutes, resulting in 20 readings over an hour. The results are shown in Fig. 5a. As this task requires invoking **Setup** once before **Collect** can be repeated as many times as needed, the time for **Setup** remains constant, irrespective of the number of subsequent collections. Conversely, the runtime overhead for completing all **Collect** steps increases linearly with the number of readings collected. We do not observe a significant runtime overhead of SLAPP_{SK} compared to Baseline_{SK} in this case.

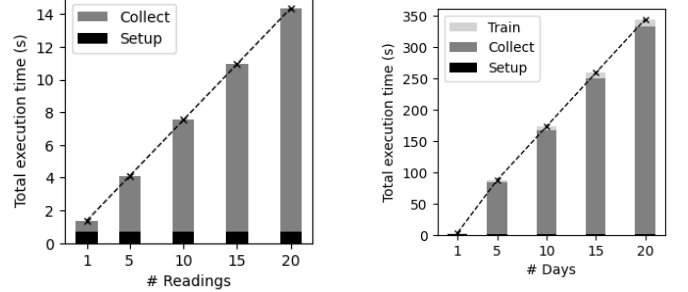
Attack Simulation. We launch the following attacks to LDP-DC⁺ in this case study:

- Adv₁ corrupts Init-State to poison the initial state.
- Adv₂ corrupts LDP-DC between **Setup** and **Collect** phases to poison raw energy readings.
- Adv₃ poisons the state in between **Setup** and **Collect** phases.
- Adv₄ poisons the final result in **Collect** phase.

LDP-DC⁺ detects all aforementioned attacks. Specifically, Adv₁ and Adv₂ are detected at the end of **Setup** and **Collect** phases, respectively. Adv₃ is caught in **Collect** since SLAPP's RoT noticed tampered state values (from CheckState) and thus refused to generate a valid proof. Similarly, Adv₄ is detected in **Collect** phase since the proof σ does not reflect the tampered output received by $\mathcal{V}rf$.

F. Case Study 2: Federated Learning

Description. Here $\mathcal{V}rf$ aims to develop an ML model for one-hour-ahead load forecasting [64]. To achieve this while ensuring user privacy, it employs FL based on LSTM [64]. As a simple embedded device, $\mathcal{P}rv$ is not designed to handle large or complex ML models due to constraints on CPU, memory, and energy. For example, the prototype board used in this case study operates at a 110MHz CPU clock speed and has only 256KB of RAM and 512KB of FLASH shared between the Secure and Non-secure Worlds. These limitations also prohibit the storage and processing of a large training dataset.


 (a) Case study 1: LDP-DC⁺

 (b) Case study 2: FL-DC⁺

Fig. 5: Sum of the runtimes of multiple data collection rounds performed over a longer period with multiple readings.

To overcome these challenges, this case study restricts each $\mathcal{P}rv$ to collecting 5 days worth of hourly energy readings (i.e., 120 data points) and training a lightweight LSTM model with a single layer of 8 neurons. Once local training is complete, the local models are transmitted to and aggregated by $\mathcal{V}rf$. FL-DC⁺ is employed to prevent poisoning.

$\mathcal{P}rv$ and $\mathcal{V}rf$ initiate FL-DC⁺ protocol by running **Setup** step, which clears all energy readings on $\mathcal{P}rv$. Next, $\mathcal{V}rf$ periodically requests **Collect** to record an energy reading into a $\mathcal{P}rv$ -local training dataset. Once $\mathcal{P}rv$ records enough energy readings, $\mathcal{V}rf$ triggers the **Train** phase by transmitting a PoSX request to $\mathcal{P}rv$. This request specifies the initial weights to be trained on, the learning rate (0.01), and the number of epochs (5). $\mathcal{P}rv$ then (provably) performs the local training. Upon completion, $\mathcal{P}rv$ sends back the updated weights along with the proof of training to $\mathcal{V}rf$. Since $\mathcal{V}rf$ and $\mathcal{P}rv$ adhere to FL-DC⁺, the protocol yields $\top$, ensuring to $\mathcal{V}rf$ that the received model was correctly trained on authentic energy data and thus can be securely aggregated onto the global model.

End-to-end Evaluation. The results of this case study are shown in Table II. Similar to the previous case study, by making the TCB application-agnostic, SLAPP_{SK} can reduce the size of Secure FLASH by a significant amount (76%). SLAPP_{SK} requires the same amount of Secure RAM regardless of the steps. In contrast, Baseline_{SK} incurs 0.7KB of Secure RAM for the **Setup** and **Collect** steps, while the **Train** step, which involves training an LSTM model, requires a more substantial amount, 32KB, of Secure RAM – around 5x of SLAPP_{SK}. These results further emphasize SLAPP's greater benefits especially when $\mathcal{F}$ is more resource-intensive, as in the case of LSTM training.

In terms of execution time, the results for the **Setup** and **Collect** steps are similar to the previous case study, with Phase 2 being the most time-consuming. However, the end-to-end execution time for **Train** is dominated by Phase 3,

which involves executing $\mathcal{F}$ (LSTM training) in the Non-Secure World. This step takes around 3 seconds, contributing $\approx 81\%$ to the end-to-end runtime.

Finally, we consider a task of applying FL-DC⁺ to collaboratively train an LSTM model on hourly energy readings collected over different numbers of days (from 1 to 20 days). The sums of total runtimes for the entire periods are shown in Fig. 5b. Since **Setup** is performed once, its overall runtime is fixed to 0.69s. As the energy sensor is read 24 times a day, the time sum for all **Collect** steps is linear with the number of days and dominates the overall runtime. The **Train** step is executed at the end of this task on all previously collected readings, resulting in a linear runtime sum ranging from 2s (over 1 day) to 12s (over 20 days). With most steps exhibiting a linear runtime, the total runtime of this task also becomes linear with the number of days.

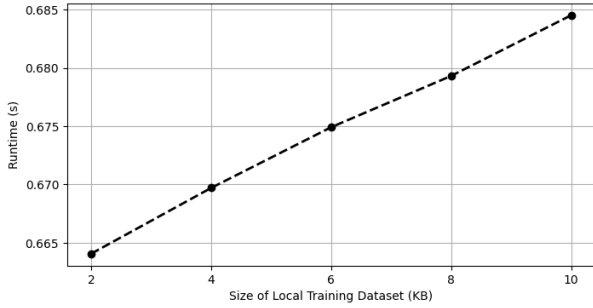


Fig. 6: $\mathcal{P}rv$ runtime overhead with varying local training sizes

Runtime Overhead vs Training Size. Next, we analyze the impact of the training dataset size on the runtime overhead introduced by FL-DC⁺. On $\mathcal{P}rv$, compared to vanilla FedAVG, FL-DC⁺ added overhead corresponds to context switches between Secure and Non-Secure Worlds plus time to hash $\mathcal{P}MEM$, authenticate $\mathcal{V}rf$ request, compute one MAC/signature, and execute one checkState/setState (one hash computation for each). Among these, only the last operation depends on the size of the training dataset, which is the PoSX state in FL-DC⁺. In this experiment, we consider larger (local) dataset sizes ranging from 2KB to 10KB. Assuming one sensor reading is collected per hour, these datasets correspond to 2.8-14 months of sensor data collection in this case study. The experimental results, shown in Fig. 6, demonstrate that SLAPP runtime overhead is linear in terms of the dataset size. Compared to the local training time of around 23s throughout this experiment, the added overhead ($\approx 0.6s$) is small: around 2.6% of this duration. We also note that since $\mathcal{V}rf$ operates independently of the training dataset, no additional runtime is incurred on $\mathcal{V}rf$.

Scalability. To evaluate the scalability of FL-DC⁺, we measure the total runtime required for $\mathcal{V}rf$ to execute this case study with a variable numbers of clients ($\mathcal{P}rv$ -s). Upon receiving PoSX-s of the **Train** phase from all clients, $\mathcal{V}rf$ performs one authentication for each PoSX and then aggregates all local models that pass the authentication checks into the current global model. We report the total runtime results on $\mathcal{V}rf$ in Fig. 7. Since the runtime linearly depends on the number of PoSX-s received, which equals the number of clients, it scales

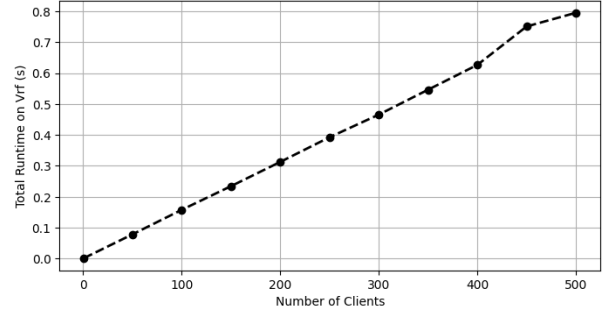


Fig. 7: Total runtime on $\mathcal{V}rf$ with multiple clients in FL-DC⁺

linearly with the number of clients. Notably, even for 500 clients, $\mathcal{V}rf$ completes its operations in less than a second.

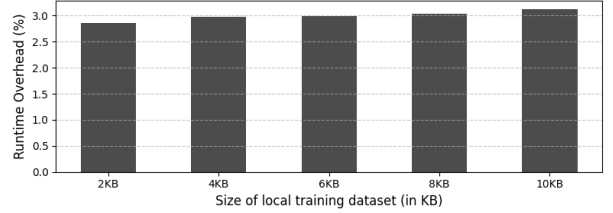


Fig. 8: FL-DC⁺ worst-case runtime overhead w.r.t. existing poisoning prevention techniques in FL

Comparison with Existing Techniques. Here, we compare the runtime of FL-DC⁺ with existing techniques for mitigating poisoning attacks in FL. We consider two types of existing techniques: the ones with modified loss functions [24], [31], [62] and the ones based on Byzantine-robust aggregation [4], [26]. In the worst case, FL-DC⁺ overhead is incurred due to three additional computations on $\mathcal{P}rv$: one for hashing $\mathcal{P}MEM$, one hash during CheckState and another during SetState³. Thus, the worst-case runtime overhead of FL-DC⁺ also depends on the size of the training dataset. To evaluate this, we measured the overhead for training datasets ranging from 2KB to 10KB. As seen in Fig. 8, our approach introduces around 3% runtime overhead.

In terms of storage overhead, SLAPP additionally requires its (application-agnostic) TCB to be implemented inside TrustZone-M Secure World. With the symmetric-key version (SLAPP_{SK}), this overhead results in 17.5KB of additional FLASH secure storage. Given its small overheads, we emphasize that FL-DC⁺ need not (and is not meant to) replace prior techniques and can be combined with them for increased security; we elaborate on this point in Section X.

Attack Simulation. We simulate two types of adversaries: $\mathcal{A}dv_D$ and $\mathcal{A}dv_M$. $\mathcal{A}dv_D$ performs data poisoning attacks to FL-DC⁺ by compromising $\mathcal{P}rv$ -local training dataset before **Train** is executed. Meanwhile, $\mathcal{A}dv_M$ simulates model poisoning by manipulating the locally trained model parameters

³Note that $\mathcal{V}rf$ authentication and the time to produce a MAC/signature are not considered FL-DC⁺ overhead since these operations are also required by existing techniques, e.g., the Byzantine-robust techniques need mutual authentication to prevent a malicious $\mathcal{P}rv$ from impersonating others and breaking the Byzantine threshold.

before the model arrives at $\mathcal{V}_{rf}$. FL-DC⁺ detects $\mathcal{A}_{adv_D}$ during the **Train** phase because SLAPP’s RoT identifies state $\mathcal{S}$ tampering ($\mathcal{S}$ is the training dataset in this case). $\mathcal{A}_{adv_M}$ is caught at the end of the **Train** as the proof does not match the tampered output.

X. INTEGRATING COMPLIMENTARY TECHNIQUES WITH SLAPP FOR ADDITIONAL BENEFITS

As mentioned in Section I, SLAPP offers two notable advantages: (1) it allows $\mathcal{P}_{rv}$ to convince $\mathcal{V}_{rf}$ that execution of a function $\mathcal{F}$ happened without any input/state assumption about $\mathcal{F}$ and (2) most SLAPP operations are performed on the client-side (i.e., $\mathcal{P}_{rv}$), making it possible to combine SLAPP with any server-side techniques. In this section, we present two concrete examples that leverage these advantages to enhance SLAPP benefits beyond poisoning protection in the scope of our system model. We note that while we focus on FL-DC⁺, the discussion in this section is also relevant to LDP-DC⁺.

First, FL-DC⁺ builds upon FedAVG algorithm, which is shown to perform poorly with non-identically distributed (IID) data across $\mathcal{P}_{rv}$ -s [76]. To cope with this, several studies have extended FedAVG to better handle non-IID data. For instance, FedProx [36] introduces a proximal term to the loss function during the local training process to help constrain local updates to be closer to the global model. As SLAPP supports PoSX for arbitrary $\mathcal{F}$, FL-DC⁺ can be adapted to incorporate FedProx by implementing the FedProx algorithm as $\mathcal{F}$ in the **Train** phase. With this minor modification, FL-DC⁺ extends poisoning prevention to the non-IID setting.

Second, SLAPP threat model focuses on software-only attacks while considering attacks that manipulate $\mathcal{P}_{rv}$ hardware or the physical environment being measured by $\mathcal{P}_{rv}$ (e.g. the ones considered in [25], [71]) out of scope. As FL-DC⁺ builds atop SLAPP, it inherits the same assumption for poisoning prevention. One common approach to address poisoning attacks in FL against hardware/physical attacks is through Byzantine-robust aggregation techniques [4], [26]. These techniques modify the aggregation step on $\mathcal{V}_{rf}$ to make it more robust against malicious updates under the Byzantine assumptions (i.e., only a certain number of $\mathcal{P}_{rv}$ -s can be compromised at a time), alleviating the impact from (but not completely preventing) hardware/physical $\mathcal{A}_{adv}$. For example, the work in [11] replaces the arithmetic mean of local gradients (as used in FedAVG) with the geometric median of means during global model updates. If hardware/physical attacks are of concern (in addition to software-based attacks), FL-DC⁺ can be adapted to support Byzantine robustness mechanisms. In particular, after receiving all verified local models from $\mathcal{P}_{rv}$ -s, $\mathcal{V}_{rf}$ in FL-DC⁺ can update the global model via Byzantine-robust aggregation rules. This helps mitigate poisoning attacks from Byzantine hardware/physical $\mathcal{A}_{adv}$ in addition to software-only $\mathcal{A}_{adv}$. We believe the combination of both approaches to significantly strengthen overall security of these schemes.

XI. RELATED WORK

Poisoning Prevention in LDP and FL. We divide existing approaches to preventing poisoning attacks into 3 categories:

data-driven, algorithmic and system-level. Data-driven approaches detect poisoning attacks based solely on the collected data without modifying the underlying collection scheme, e.g., by applying normalization [7] and analyzing distances between or error rates from data [6], [37], [23]. Meanwhile, algorithmic approaches modify the data collection algorithm to make it resilient against poisoning attacks, e.g., by incorporating with Byzantine fault-tolerant techniques [4], [26], [72] or using modified loss functions [24], [31], [62]. Finally, system-level approaches [59] leverage $\mathcal{P}_{rv}$ ’s security architecture to mitigate poisoning attacks. SLAPP falls into the last category.

Based on these categories, we qualitatively compare SLAPP with current defenses in Table III. It shows that SLAPP is the only approach that offers strong robustness against adaptive adversaries without making assumptions about Byzantine $\mathcal{A}_{adv}$ (i.e., SLAPP allows $\mathcal{A}_{adv}$ to corrupt any number of $\mathcal{P}_{rv}$ -s) or data distributions (i.e., SLAPP allows each $\mathcal{P}_{rv}$ to have independent data distributions). As SLAPP can deterministically discern poisoned data from the benign, it results in no utility loss on aggregate data. It also supports both LDP and FL-based data collection schemes. As a system-level approach, SLAPP requires $\mathcal{P}_{rv}$ to have an RoT, which is becoming more common in modern IoT devices. Moreover, SLAPP’s RoT hosts only a small TCB agnostic to the data collection scheme (FL or LDP); this enables a *one-and-done* process for validating its correctness (e.g., through formal verification, which is an interesting avenue for future work). Besides, to the best of our knowledge, none of existing work provides PoSX-equivalent guarantee of provable integrity all the way from data acquisition until its *de facto* usage as an aggregated statistical result or global ML model. They also do not address the issues of input validation or state preservation for arbitrary functions $\mathcal{F}$ to be executed on simple $\mathcal{P}_{rv}$ devices.

Verifiable Software Integrity in Embedded Devices. To secure low-end embedded devices, various low-cost security architectures have been proposed for remote verification of their software state via integrity proofs [52]. These proofs vary in terms of expressiveness, with simpler ones confirming correct binary presence (remote attestation) [12], [21], [5], [33], [20], [50], while more expressive ones support verification of arbitrary code execution. Aside from PoX architectures [13], [8], [54], [39], control flow attestation/auditing (CFA) techniques [48], [1], [65], [75], [63], [70], [18], [17], [74], [14], [9] prove to $\mathcal{V}_{rf}$ the exact order in which instructions have executed within a particular code in $\mathcal{P}_{rv}$, thus enabling detection of code reuse attacks that can be triggered if the code whose execution is being proven is itself vulnerable. Data flow attestation [63], [53], [17] augments CFA to generate evidence about memory safety violations even when exploits do not alter a program’s legal control flow path.

Private Data Collection on Edge Devices. Complementary to privacy mechanisms focusing on hiding private data from back-ends (e.g., LDP/FL), recent work has delved into assuring that private data is secure against compromised sensing devices “from its birth”, i.e., from the moment when it is digitized. VERSA [51] was proposed as a HW/SW architecture to guarantee that only the correct execution of expected and explicitly authorized software can access and manipulate sens-

TABLE III: Comparison with current defenses (○, ● and ● indicates the degree of support/assumption/impact)

Feature (→) Approach (↓)	Adv		Data		Application		Prv	
	Adaptive Robustness	Byzantine Assumption	Distribution Assumption	Impact on Agg. Utility	Support DP/LDP	Support FL/ML	Require RoT	App-indep. RoT
Data-driven	Sniper [6]	○	●	●	○	●	○	N/A
	Normalize. [7]	●	●	●	●	○	○	N/A
	ERR+LFR [23]	○	●	●	○	●	○	N/A
	Clustering [37]	○	●	●	●	○	○	N/A
Algorithmic	Multi-Krum [4]	○	●	●	○	●	○	N/A
	RoLR [24]	○	●	●	○	●	○	N/A
	TRIM [31]	○	●	○	○	●	○	N/A
	FL-WBC [62]	●	○	○	○	●	●	○
System-level	CrowdGuard [59]	●	○	○	○	●	●	○
	EBFA [72]	○	●	○	○	●	●	○
	SLAPP (this work)	●	○	○	●	●	●	●

ing interfaces. As a consequence, it blocks malware/modified software from accessing sensitive sensed quantities by default. Following this notion, Sensing And Actuation As A Privilege (SA4P) [15], [16] realizes this concept using ARM TrustZone.

System-level Approaches in FL. Besides poisoning prevention, several system-level approaches have been proposed to provide different security and privacy guarantees in FL for higher-end *Prv*-s. In terms of privacy, PPFL [42] introduces a layer-wise training method within a Trusted Execution Environment (TEE) to provide confidentiality of training data while adhering to the memory constraints of the TEE. GradSec [41] improves upon this approach by significantly reducing the runtime overhead associated with the training process. Also, other approaches such as EBFA [72], CrowdGuard [59] and Hashemi et al. [27] propose the use of TEE to protect privacy of the training data/model while running poisoning prevention techniques outside the TEE. For security, Pelta [56] leverages TEE to mitigate adversarial (a.k.a. evasion) attacks in FL. It securely hides critical model parameters and updates these parameters (i.e., backpropagation) inside the client-side TEE. As a result, it limits a malicious client's access to only a partial model, making it harder to craft adversarial examples. While sharing the similarity of leveraging TEE, these approaches do not use TEE to address data/model poisoning attacks in FL, which is one of the focal points in this work.

XII. CONCLUSION

We defined and developed stateful proofs of execution, a system security primitive to thwart poisoning in applications such as differential privacy and federated learning. We analyze the security of our design (SLAPP) and evaluate its performance with an open-source prototype. Results indicate strong poisoning prevention guarantees at modest overhead applicable even to MCU-based resource-constrained IoT devices.

ACKNOWLEDGEMENTS

We sincerely thank TIFS anonymous reviewers for their constructive feedback. Norrathep Rattanavipanon was supported by the National Science, Research and Innovation Fund (NSRF) and Prince of Songkla University (Grant No. COC6701016S) and the NSRF via the Program Management Unit for Human Resources & Institutional Development, Research and Innovation [grant number B13F670122].

REFERENCES

- [1] Tigist Abera, N Asokan, Lucas Davi, Jan-Erik Ekberg, Thomas Nyman, Andrew Paverd, Ahmad-Reza Sadeghi, and Gene Tsudik. C-flat: control-flow attestation for embedded systems software. In *ACM CCS*, 2016.
- [2] Mahmoud Ammar, Adam Caulfield, and Ivan De Oliveira Nunes. Sok: Integrity, attestation, and auditing of program execution. In *IEEE Symposium on Security and Privacy (SP)*, 2025.
- [3] William A Arbaugh, David J Farber, and Jonathan M Smith. A secure and reliable bootstrap architecture. In *IEEE Symposium on Security and Privacy*, 1997.
- [4] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. *Advances in neural information processing systems*, 30, 2017.
- [5] Ferdinand Brasser, Brahim El Mahjoub, Ahmad-Reza Sadeghi, Christian Wachsmann, and Patrick Koeberl. Tytan: Tiny trust anchor for tiny devices. In *DAC*, 2015.
- [6] Di Cao, Shan Chang, Zhijian Lin, Guohua Liu, and Donghong Sun. Understanding distributed poisoning attack in federated learning. In *IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, 2019.
- [7] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Data poisoning attacks to local differential privacy protocols. In *USENIX Security Symposium*, 2021.
- [8] Adam Caulfield, Norrathep Rattanavipanon, and Ivan De Oliveira Nunes. Asap: Reconciling asynchronous real-time operations and proofs of execution in simple embedded systems. In *ACM/IEEE Design Automation Conference (DAC)*, 2022.
- [9] Adam Caulfield, Norrathep Rattanavipanon, and Ivan De Oliveira Nunes. Acfa: Secure runtime auditing & guaranteed device healing via active control flow attestation. In *USENIX Security Symposium*, 2023.
- [10] Z Berkay Celik, Patrick McDaniel, and Gang Tan. Soteria: Automated IoT safety and security analysis. In *USENIX ATC 18*, 2018.
- [11] Yudong Chen, Lili Su, and Jiaming Xu. Distributed statistical machine learning in adversarial settings: Byzantine gradient descent. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 1(2):1–25, 2017.
- [12] Ivan De Oliveira Nunes, Karim Eldefrawy, Norrathep Rattanavipanon, Michael Steiner, and Gene Tsudik. VRASED: A verified hardware/software co-design for remote attestation. In *USENIX Security Symposium*, 2019.
- [13] Ivan De Oliveira Nunes, Karim Eldefrawy, Norrathep Rattanavipanon, and Gene Tsudik. APEX: A verified architecture for proofs of execution on remote devices under full software compromise. In *USENIX Security Symposium*, 2020.
- [14] Ivan De Oliveira Nunes, Sashidhar Jakkamsetti, and Gene Tsudik. Tiny-cfa: Minimalistic control-flow attestation using verified proofs of execution. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021.
- [15] Piet De Vaere. *Fine-Grained Access Control For Sensors, Actuators, and Automation Networks*. PhD thesis, ETH Zurich, 2023.
- [16] Piet De Vaere, Felix Stöger, Adrian Perrig, and Gene Tsudik. The sa4p framework: Sensing and actuation as a privilege. *ACM AsiaCCS*, 2024.
- [17] Ghada Dessouky, Tigist Abera, Ahmad Ibrahim, and Ahmad-Reza Sadeghi. Litehax: lightweight hardware-assisted attestation of program execution. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018.

- [18] Ghada Dessouky, Shaza Zeitouni, Thomas Nyman, Andrew Paverd, Lucas Davi, Patrick Koeberl, N Asokan, and Ahmad-Reza Sadeghi. Lo-fat: Low-overhead control flow attestation in hardware. In *ACM/IEEE Design Automation Conference (DAC)*, 2017.
- [19] Bolin Ding et al. Collecting telemetry data privately. *Advances in Neural Information Processing Systems*, 30, 2017.
- [20] Karim Eldefrawy, Norrathep Rattanavipanon, and Gene Tsudik. Hydra: hybrid design for remote attestation (using a formally verified microkernel). In *ACM WiSec*, 2017.
- [21] Karim Eldefrawy, Gene Tsudik, Aurélien Francillon, and Daniele Perito. SMART: Secure and minimal architecture for (establishing dynamic) root of trust. In *NDSS*, 2012.
- [22] Úlfar Erlingsson, Vasily Pihur, and Aleksandra Korolova. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *ACM SIGSAC conference on computer and communications security*, 2014.
- [23] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Gong. Local model poisoning attacks to Byzantine-Robust federated learning. In *USENIX Security Symposium*, 2020.
- [24] Jiashi Feng, Huan Xu, Shie Mannor, and Shuicheng Yan. Robust logistic regression and classification. *Advances in neural information processing systems*, 2014.
- [25] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access*, 7:47230–47244, 2019.
- [26] Rachid Guerraoui, Sébastien Rouault, et al. The hidden vulnerability of distributed learning in byzantium. In *International Conference on Machine Learning*, 2018.
- [27] Hanieh Hashemi, Yongqin Wang, Chuan Guo, and Murali Annavaram. Byzantine-robust and privacy-preserving framework for fedml. *arXiv preprint arXiv:2105.02295*, 2021.
- [28] Joanne Hinds, Emma J Williams, and Adam N Joinson. “it wouldn’t happen to me”: Privacy concerns and perspectives following the cambridge analytica scandal. *International Journal of Human-Computer Studies*, 143:102498, 2020.
- [29] François Hublet, David Basin, and Srdjan Krstić. Enforcing the gdpr. In *European Symposium on Research in Computer Security*, 2023.
- [30] Andreas Hülsing, Joost Rijnveld, and Peter Schwabe. Armed sphincs: Computing a 41 kb signature in 16 kb of ram. In *IACR International Conference on Practice and Theory in Public-Key Cryptography*, 2016.
- [31] Matthew Jagielski, Alina Oprea, Battista Biggio, Chang Liu, Cristina Nita-Rotaru, and Bo Li. Manipulating machine learning: Poisoning attacks and countermeasures for regression learning. In *2018 IEEE symposium on security and privacy (SP)*, 2018.
- [32] Scott Jordan, Yoshimichi Nakatsuka, Ercan Ozturk, Andrew Paverd, and Gene Tsudik. Viceroy: Gdpr/ccpa-compliant enforcement of verifiable accountless consumer requests. 2023.
- [33] Patrick Koeberl, Steffen Schulz, Ahmad-Reza Sadeghi, and Vijay Varadharajan. TrustLite: A security architecture for tiny embedded devices. In *European Conference on Computer Systems*, 2014.
- [34] Jakub Konečný, H Brendan McMahan, Felix X Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. Federated learning: Strategies for improving communication efficiency. *arXiv preprint arXiv:1610.05492*, 2016.
- [35] He Li, Lu Yu, and Wu He. The impact of gdpr on global technology development. *Journal of Global Information Technology Management*, 22(1):1–6, 2019.
- [36] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems*, 2020.
- [37] Xiaoguang Li, Ninghui Li, Wenhui Sun, Neil Zhenqiang Gong, and Hui Li. Fine-grained poisoning attack to local differential privacy protocols for mean and variance estimation. In *USENIX Security Symposium*, 2023.
- [38] Zhuoran Ma, Jianfeng Ma, Yinbin Miao, Yingjiu Li, and Robert H Deng. Shieldfl: Mitigating model poisoning attacks in privacy-preserving federated learning. *IEEE Transactions on Information Forensics and Security*, 17:1639–1654, 2022.
- [39] Jonathan M McCune, Bryan J Parno, Adrian Perrig, Michael K Reiter, and Hiroshi Isozaki. Flicker: An execution infrastructure for tcb minimization. In *ACM EuroSys*, 2008.
- [40] Patrick McDaniel and Stephen McLaughlin. Security and privacy challenges in the smart grid. *IEEE security & privacy*, 7(3):75–77, 2009.
- [41] Aghiles Ait Messaoud, Sonia Ben Mokhtar, Vlad Nitu, and Valerio Schiavoni. Shielding federated learning systems against inference attacks with arm trustzone. In *ACM/IFIP International Middleware Conference*, 2022.
- [42] Fan Mo, Hamed Haddadi, Kleomenis Katevas, Eduard Marin, Diego Perino, and Nicolas Kourtellis. Ppfl: Privacy-preserving federated learning with trusted execution environments. In *Annual international conference on mobile systems, applications, and services*, 2021.
- [43] Soumya Ranjan Moharana, Vijay Kumar Jha, Anurag Satpathy, Sourav Kanti Addya, Ashok Kumar Turuk, and Banshidhar Majhi. Secure key-distribution in iot cloud networks. In *International Conference on Sensing, Signal Processing and Security (ICSSS)*, 2017.
- [44] Andrés Molina-Markham, Prashant Shenoy, Kevin Fu, Emmanuel Cecchet, and David Irwin. Private memoirs of a smart meter. In *ACM workshop on embedded sensing systems for energy-efficiency in building*, 2010.
- [45] Max Mössinger, Benedikt Petschkuhn, Johannes Bauer, Ralf C Staude-meyer, Marcin Wójcik, and Henrich C Pöhls. Towards quantifying the cost of a secure iot: Overhead and energy consumption of ecc signatures on an arm-based device. In *IEEE International Symposium on A World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, 2016.
- [46] National Vulnerability Database. CVE-2017-14201, 2017. Accessed: 2024-10-10.
- [47] National Vulnerability Database. CVE-2020-10023, 2020. Accessed: 2024-10-10.
- [48] Antonio Joia Neto and Ivan De Oliveira Nunes. Isc-flat: On the conflict between control flow attestation and real-time operations. In *RTAS*, 2023.
- [49] Ruben Niederhagen, Johannes Roth, and Julian Wälde. Streaming sphincs+ for embedded devices using the example of tpm. In *International Conference on Cryptology in Africa*, 2022.
- [50] Job Noorman, Jo Van Bulck, Jan Tobias Mühlberg, Frank Piessens, Pieter Maene, Bart Preneel, Ingrid Verbauwhede, Johannes Götzfried, Tilo Müller, and Felix Freiling. Sancus 2.0: A low-cost security architecture for iot devices. *ACM TOPS*, 20(3):1–33, 2017.
- [51] Ivan De Oliveira Nunes, Seoyeon Hwang, Sashidhar Jakkamsetti, and Gene Tsudik. Privacy-from-birth: Protecting sensed data from malicious sensors with versa. In *IEEE Symposium on Security and Privacy*, 2022.
- [52] Ivan De Oliveira Nunes, Sashidhar Jakkamsetti, Norrathep Rattanavipanon, and Gene Tsudik. Towards remotely verifiable software integrity in resource-constrained iot devices. *IEEE Communications Magazine*, 2024.
- [53] Ivan De Oliveira Nunes, Sashidhar Jakkamsetti, and Gene Tsudik. Dialed: Data integrity attestation for low-end embedded devices. In *ACM/IEEE Design Automation Conference (DAC)*, 2021.
- [54] Avani Dave Nilanjan Banerjee Chintan Patel. Rares: Runtime attack resilient embedded system design using verified proof-of-execution. *arXiv preprint arXiv:2305.03266*, 2023.
- [55] Sandro Pinto and Nuno Santos. Demystifying arm trustzone: A comprehensive survey. *ACM computing surveys (CSUR)*, 51(6):1–36, 2019.
- [56] Simon Queyruet, Valerio Schiavoni, and Pascal Felber. Mitigating adversarial attacks in federated learning with trusted execution environments. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2023.
- [57] Norrathep Rattanavipanon and Ivan De Oliveira Nunes. Slapp repo. <https://github.com/norrathep/SLAPP>.
- [58] Srivaths Ravi, Anand Raghunathan, and Srimat Chakradhar. Tamper resistance mechanisms for secure embedded systems. In *VLSI Design*, 2004.
- [59] Phillip Rieger, Torsten Krauß, Markus Miettinen, Alexandra Dmitrienko, and Ahmad-Reza Sadeghi. Crowdguard: Federated backdoor detection in federated learning. *arXiv preprint arXiv:2210.07714*, 2022.
- [60] Tjerand Silde. Comparative study of ecc libraries for embedded devices. *Norwegian University of Science and Technology, Tech. Rep.*, 2019.
- [61] STMicroelectronics. Nucleo-l552ze-q. <https://estore.st.com/en/nucleo-l552ze-q-cpn.html>.
- [62] Jingwei Sun, Ang Li, Louis DiValentin, Amin Hassanzadeh, Yiran Chen, and Hai Li. Fl-wbc: Enhancing robustness against model poisoning attacks in federated learning from a client perspective. *Advances in Neural Information Processing Systems*, 2021.
- [63] Zhichuang Sun, Bo Feng, Long Lu, and Somesh Jha. Oat: Attesting operation integrity of embedded devices. In *IEEE S&P*, 2020.
- [64] Afaf Taïk and Soumaya Cherkaoui. Electrical load forecasting using edge computing and federated learning. In *IEEE international conference on communications (ICC)*, 2020.
- [65] Flavio Toffalini, Eleonora Losiouk, Andrea Biondo, Jianying Zhou, and Mauro Conti. ScaRR: Scalable runtime remote attestation for complex systems. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2019.
- [66] Vale Tolpegin, Stacey Truex, Mehmet Emre Gursay, and Ling Liu. Data poisoning attacks against federated learning systems. In *ESORICS*, 2020.

- [67] Florian Tramer, Nicholas Carlini, Wieland Brendel, and Aleksander Madry. On adaptive attacks to adversarial example defenses. *Advances in neural information processing systems*, 2020.
- [68] Trusted Computing Group. Trusted platform module (tpm). <http://www.trustedcomputinggroup.org/work-groups/trusted-platform-module/>, 2017.
- [69] Gene Tsudik. Staving off the iot armageddon. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 2–3, 2024.
- [70] Jinwen Wang, Yujie Wang, Ao Li, Yang Xiao, Ruide Zhang, Wenjing Lou, Y Thomas Hou, and Ning Zhang. ARI: Attestation of real-time mission execution integrity. In *USENIX Security Symposium*, 2023.
- [71] Emily Wenger, Josephine Passananti, Arjun Nitin Bhagoji, Yuanshun Yao, Haitao Zheng, and Ben Y Zhao. Backdoor attacks against deep learning systems in the physical world. In *IEEE/CVF conference on computer vision and pattern recognition*, 2021.
- [72] Jingyi Yao, Chen Song, Hongjia Li, Yuxiang Wang, Qian Yang, and Liming Wang. An enclave-aided byzantine-robust federated aggregation framework. In *IEEE Wireless Communications and Networking Conference (WCNC)*, 2024.
- [73] Razieh Nokhbeh Zaeem and K Suzanne Barber. The effect of the gdpr on privacy policies: Recent progress and future promise. *ACM Transactions on Management Information Systems (TMIS)*, 12(1):1–20, 2020.
- [74] Shaza Zeitouni, Ghada Dessouky, Orlando Arias, Dean Sullivan, Ahmad Ibrahim, Yier Jin, and Ahmad-Reza Sadeghi. Atrium: Runtime attestation resilient under memory attacks. In *ICCAD*, 2017.
- [75] Yumei Zhang, Xinzhi Liu, Cong Sun, Dongrui Zeng, Gang Tan, Xiao Kan, and Siqi Ma. ReCFA: resilient control-flow attestation. In *Annual Computer Security Applications Conference*, 2021.
- [76] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. Federated learning with non-iid data. *arXiv preprint arXiv:1806.00582*, 2018.



Norrathep Rattavipanon received his Ph.D. in Computer Science from the University of California, Irvine in 2019. Currently, he is an Assistant Professor with the College of Computing, Prince of Songkla University, Phuket Campus. His research interests lie in the area of security and privacy, particularly in embedded systems and IoT security, software and binary analysis, and security/privacy in machine learning systems.



Ivan De Oliveira Nunes is an Assistant Professor at the University of Zurich (UZH) where he leads the Security and Privacy of Information, Systems and Networks (SPINS) group within the Department of Informatics. Prior to joining UZH, he was an Assistant Professor at the Rochester Institute of Technology. He earned his Ph.D. from the University of California, Irvine. His research interests include Security & Privacy, Computer Networking, Computing Systems, and particularly the intersections of these fields.